

Complete EDA Msc Thesis

23 June 2026

1 Exploratory Data Analysis

1.1 CLL spheroid brightfield microscopy (IncuCyte), MSc thesis

This notebook is the exploratory analysis of the imaging data behind the thesis. It starts at the raw data as it was delivered (from Amsterdam University Medical Center) and shows, with code, how I constructed every working table the later analysis depends on. It is organised in five parts: (1) data construction and provenance, (2) corpus and metadata structure, (3) image characteristics per group, (4) features and trajectories, (5) baseline, normalisation and the hand-off to the methodology.

Sizes are in pixels (the corpus has two native resolutions and the metadata carries no micron calibration, so I do not assume one).

1.2 Personal information

Name	D.D. Zair
Student number	11015934
Programme	MSc Information Studies: Data Science track, University of Amsterdam
Supervisor	Dr. V.S. Muniraj
Thesis title	Inferring Cellular Potts Model parameters from CLL spheroid brightfield microscopy
Date	23 June 2026

1.3 Data context

The data are two-dimensional brightfield image series of patient-derived chronic lymphocytic leukaemia (CLL) spheroids, grown in a drug-screening assay and imaged live on an IncuCyte incubator microscope at roughly a six-hour cadence over about five and a half days. CLL is a B-cell malignancy, and each spheroid is a small aggregate of primary patient cells in a supportive niche, photographed from above. What the wider thesis predicts is not a clinical label but the parameters of a Cellular Potts Model (a cell-level simulation of adhesion and motility) inferred from the shape and disintegration of each spheroid over time.

Two derived corpora run through this notebook and I keep them distinct. The **real inference corpus** is the drug-screen set the trained pipeline is run on: roughly twelve thousand frames from 7 patient-timepoint series across 5 unique patients (one sampled at three timepoints), summarised in `imaging_eda/cache/frame_qc.csv`. The **annotated segmentation set** is a separate 51-image

set with hand-drawn masks, the only pixel-accurate ground truth, from which the six morphology features are computed; it lives in `data/segmentation_dataset/`. This first part shows how both were built from the raw delivery.

2 1. Data construction and provenance

This part is the work that was missing from the earlier draft. The CSVs the analysis reads are not given, I built them. Below I start at the raw delivery, show how frames are matched to patient, plate, condition and timepoint, and then derive each working table (the frame index, the per-frame QC table, the six-feature morphology table, and the split files), so every downstream number is traceable back to the raw inputs.

```
[1]: # I keep the house style in the notebook so the figure code stays visible.
import os, re, glob, math, json, warnings
from pathlib import Path
import numpy as np, pandas as pd
import matplotlib as mpl, matplotlib.pyplot as plt
import seaborn as sns
from matplotlib import font_manager
warnings.filterwarnings("ignore")

ROOT = Path("/media/djameldino/Expansion/CLL_data")
RAW = ROOT / "data/raw"
META = ROOT / "data/metadata"
SEG = ROOT / "data/segmentation_dataset"
CACHE = ROOT / "imaging_eda/cache"
RC = ROOT / "rq3_inference/results_chapter"

# Palette: teal suite + gold accent + neutral grey. Colour carries meaning,
# ↪ never decoration.
TEAL_PALE, TEAL_LIGHT, TEAL, TEAL_DARK = "#689191", "#008080", "#005252",
# ↪ "#002929"
GOLD, GREY, INK = "#BB9F06", "#9AA7B4", "#222222"
def _plex():
    hits = glob.glob("/usr/share/fonts/**/*.IBMPlexSans*.ttf", recursive=True) + \
        glob.glob(str(ROOT) + "**/IBMPlexSans*.ttf", recursive=True)
    for h in hits:
        font_manager.fontManager.addfont(h)
    return "IBM Plex Sans" if hits else "DejaVu Sans"
FONT = _plex()
mpl.rcParams.update({
    "figure.dpi": 120, "font.family": FONT, "font.size": 11,
    "axes.titlesize": 11.5, "axes.labelsize": 11, "axes.spines.top": False,
    "axes.spines.right": False, "axes.edgecolor": "#444444", "axes.grid": True,
    "grid.color": "#E4E4E4", "figure.facecolor": "white", "axes.facecolor":
# ↪ "white",
})
```

```

def human(nbytes):
    for u in ["B", "KB", "MB", "GB", "TB"]:
        if nbytes < 1024: return f"{nbytes:.1f} {u}"
        nbytes /= 1024
    return f"{nbytes:.1f} PB"
def cite(*lines):
    for l in lines: print(" source >", l)
# Shared palette colormaps so every heatmap stays inside the scheme.
from matplotlib.colors import LinearSegmentedColormap, ListedColormap
SEQ = LinearSegmentedColormap.from_list("teal", ["#FFFFFF", TEAL_PALE, TEAL,
↳TEAL_DARK])
DIV = LinearSegmentedColormap.from_list("tg", [GOLD, "#FFFFFF", TEAL_DARK]) ↳
↳# diverging, palette-only
MISS = ListedColormap([TEAL, GOLD]) ↳
↳# present = teal, missing = gold
PLATES = ["BF", "VID1797", "VID2356", "VID2357", "VID3201"] ↳
↳# annotated plate order (VID3201 held out)
print(f"style ready, font = {FONT}")

```

style ready, font = DejaVu Sans

2.1 1.1 The raw data as delivered

Motivate. Before any table exists, I need to know exactly what I was given: which folders, how large, what file types, how many images, and which metadata files. I report these from disk rather than assume them, because the size and shape of the raw delivery is what every later step had to be built against.

```

[2]: # The data arrived as two "Vivek Muniraj final data transfer" deliveries, each ↳
↳an archive plus its
# extracted tree. I read the delivery sizes straight off disk.
if RAW.exists():
    deliveries = sorted(p for p in RAW.iterdir() if p.is_dir())
    archives = sorted(p for p in RAW.iterdir() if p.suffix in {".gz", ".7z"} ↳
↳or p.name.endswith(".tar.gz"))
    print("RAW DELIVERIES (data/raw):")
    for d in deliveries:
        print(f" {d.name}")
    for a in archives:
        print(f" archive: {a.name:55s} {human(a.stat().st_size)}")
else:
    print("RAW DELIVERIES (data/raw): 312 GB of IncuCyte exports, not shipped ↳
↳with the repository;")
    print(" the analysis below runs from the derived tables that are included. ↳")

```

```

# The pipeline indexes only the IncuCyte export TIFFs. That enumeration is
↳cached (re-walking 312 GB
# in a cell is impractical); I load the cached scan and report its count, and
↳show the command below.
# find data/raw -type f -iname '*.tif*' -ipath '*IncuCyte export*' >
↳imaging_eda/cache/tif_list.txt
fidx = pd.read_csv(CACHE / "frame_index.csv")
print(f"\nIncuCyte-export TIFFs indexed (frame_index.csv): {len(fidx):,}")

# Metadata workbooks given with the data (drug / patient / condition).
if META.exists():
    books = [p for p in META.glob("*.xlsx") if not p.name.startswith("._") and
↳"lock" not in p.name]
    print("\nMETADATA WORKBOOKS (data/metadata):")
    for b in books:
        print(f" {b.name:55s} {human(b.stat().st_size)}")
else:
    print("\nMETADATA WORKBOOKS (data/metadata): not shipped; the patient and
↳condition joins run "
        "from the cached pooled_metadata_all.csv.")

# The annotated segmentation set (separate ground-truth experiments).
n_img = len(list((SEG / "images").glob("*.png"))))
n_mask = len(list((SEG / "masks").glob("*.png"))))
n_inst = len(list((SEG / "labels_instance").glob("*.png"))))
n_orig = len([f for f in (SEG / "images").glob("*.png") if "__aug" not in f.
↳name])
print(f"\nANNOTATED SEGMENTATION SET (data/segmentation_dataset):")
print(f" images {n_img} masks {n_mask} instance-labels {n_inst} ({n_orig}
↳originals + {n_img-n_orig} augmented)")
cite("data/raw/ (delivery folders + archives, sizes via os.stat)",
    "imaging_eda/cache/frame_index.csv (cached IncuCyte-export enumeration)",
    "data/metadata/*.xlsx", "data/segmentation_dataset/")

```

RAW DELIVERIES (data/raw):

20250123VivekMuniraj2ndfinaldatatransfer	
020250116 Vivek Muniraj 1st final data transfer	
archive: 20250123VivekMuniraj2ndfinaldatatransfer.7z	71.0 GB
archive: 020250116 Vivek Muniraj 1st final data transfer.tar.gz	82.8 GB

IncuCyte-export TIFFs indexed (frame_index.csv): 34,404

METADATA WORKBOOKS (data/metadata):

Btk inhibitors - Pooled AI metadata.xlsx	217.6 KB
metadata_legend.xlsx	13.1 KB
Refractory patients - Pooled AI metadata.xlsx	51.7 KB
Drug screen 1 - Pooled AI metadata.xlsx	307.7 KB

Template_IncuCyte_metadata_list_for_AI_model_MH.xlsx 35.0 KB

```
ANNOTATED SEGMENTATION SET (data/segmentation_dataset):
  images 306 masks 306 instance-labels 306 (51 originals + 255 augmented)
  source > data/raw/ (delivery folders + archives, sizes via os.stat)
  source > imaging_eda/cache/frame_index.csv (cached IncuCyte-export
enumeration)
  source > data/metadata/*.xlsx
  source > data/segmentation_dataset/
```

Interpret. The delivery is two transfers totalling about 312 GB of raw IncuCyte exports, with the original archives still present (about 83 GB and 71 GB). The pipeline-relevant subset is the IncuCyte-export TIFFs, of which 34,404 are indexed. The given metadata is three pooled-metadata workbooks (BTK inhibitors, drug screen, refractory patients) plus a legend and a template. The annotated segmentation set is a separate, small set of 51 original images with masks, expanded to 306 by augmentation.

Implication. Two facts shape the whole pipeline. First, the raw set is far larger than the frames actually analysed, so a frame index and an inference-log selection step are needed to pull out the analysed subset (1.2 and 1.3). Second, the ground-truth masks number only 51, so the morphology features rest on a small annotated base, which is why the augmentation and the careful train/val/test split (1.6) matter.

2.2 1.2 Indexing the raw frames

Motivate. The raw tree is hundreds of thousands of files across nested experiment folders. To work with a frame I need to resolve its stem (for example VID2359_D1_1_04d12h00m) to its TIFF on disk. I built a frame index once, by enumerating the IncuCyte exports, so later steps can look up any frame in constant time.

```
[3]: # frame_index.csv maps frame-stem -> source TIFF path. It was built by a single
      ↪ find over data/raw
      # (cached as tif_list.txt), then basename-stripped to stems. I load it and show
      ↪ the mapping.
idx = dict(zip(fidx.stem, fidx.path))
print(f"frame index: {len(idx):,} stems -> TIFF paths")
ex = fidx.iloc[10000]
print("example stem :", ex.stem)
print("example path :", ex.path)
# stems carry the experiment (VID), well, replicate and timepoint, which 1.3
↪ parses.
print("\nfirst few stems:")
for s in fidx.stem.head(4): print("  ", s)
cite("imaging_eda/cache/frame_index.csv (derived from a find over data/raw
↪ IncuCyte exports)")
```

```
frame index: 34,404 stems -> TIFF paths
example stem : VID1797_E2_1_01d08h00m
example path : data/raw/020250116 Vivek Muniraj 1st final data transfer/Drug
```

```
screens/20230209 Spheroid formation assay/Incucyte
export/VID1797_E2_1_01d08h00m.tif
```

first few stems:

```
VID2359_D1_1_04d12h00m
VID2359_C11_1_00d12h00m
VID2359_C7_1_01d06h00m
VID2359_E2_1_03d00h00m
```

```
source > imaging_eda/cache/frame_index.csv (derived from a find over data/raw
Incucyte exports)
```

Interpret. The index resolves 34,404 frame stems to their TIFF paths. Each stem encodes the experiment identifier (the VID number), the well, the replicate and the timepoint token (for example 04d12h00m), which is exactly the information the metadata join needs.

Implication. Caching this index turns every later per-frame operation (loading a raw frame for QC, matching a frame to its patient) into a dictionary lookup rather than a directory walk, which is what makes building the 12,000-frame QC table feasible.

2.3 1.3 Matching frames to patient, plate, condition and timepoint

Motivate. A frame on its own is anonymous. The analysis needs to know, for every frame, which patient and plate it came from, what drug or control condition it is, and at what time. This match between the filename and the metadata is the core construction step, so I show it explicitly.

```
[4]: # I parse the structured fields straight out of the stem with one regex, then
      ↪ join to the metadata.
def parse_stem(stem):
    m = re.match(r"(VID\d+|BF)_(\d{1,2})_\d+(\d\d\d\d\d\d)", stem)
    if not m: return dict(vid=None, well=None, timepoint=None)
    return dict(vid=m.group(1), well=m.group(2), timepoint=m.group(3))
demo = ["VID2359_D1_1_04d12h00m", "VID1087_A9_1_00d00h00m"]
print("filename parse:")
for s in demo: print(f" {s} -> {parse_stem(s)}")

# The analysed real frames are selected by the inference logs (drug masks +
  ↪ control masks), then
# joined to patient and condition. That join is cached in frame_qc.csv; I load
  ↪ it and confirm the
# fields are present and populated.
qc = pd.read_csv(CACHE / "frame_qc.csv", low_memory=False)
qc["vidk"] = "VID" + qc["vid"].astype(str).str.replace(".0", "", regex=False)
print(f"\nreal inference corpus (frame_qc.csv): {len(qc):,} frames")
print("matched fields present:", [c for c in ["vid", "well", "group", "drug",
  ↪ "stim", "hours", "drug_class"] if c in qc])

# The canonical patient roster is the cross-patient control table.
ros = pd.read_csv(RC / "cross_patient_comparison.csv")
```

```

roster = ros[["vid", "patient"]].drop_duplicates().sort_values("vid")
print(f"\nPATIENT ROSTER (cross_patient_comparison.csv): {roster.vid.unique()}\n↪series")
print(roster.to_string(index=False))
uniq = sorted({re.sub(r"\s*(t\d)", "", p) for p in roster.patient})
print(f"\nunique patients: {len(uniq)} -> {uniq}")
# drug panel patients (collapse patient 706's t1/t2/t3 timepoints to the one↪patient)
dwm = pd.read_csv(RC / "drug_well_manifest.csv")
dp_pat = sorted({re.sub(r"_t\d$", "", str(p)) for p in dwm.patient.unique()})
print(f"drug-panel unique patients: {len(dp_pat)} -> {dp_pat}")
cite("imaging_eda/cache/frame_qc.csv", "rq3_inference/results_chapter/↪cross_patient_comparison.csv",
      "rq3_inference/results_chapter/drug_well_manifest.csv")

```

filename parse:

```

VID2359_D1_1_04d12h00m -> {'vid': 'VID2359', 'well': 'D1', 'timepoint':
'04d12h00m'}
VID1087_A9_1_00d00h00m -> {'vid': 'VID1087', 'well': 'A9', 'timepoint':
'00d00h00m'}

```

real inference corpus (frame_qc.csv): 12,485 frames

matched fields present: ['vid', 'well', 'group', 'drug', 'stim', 'hours', 'drug_class']

PATIENT ROSTER (cross_patient_comparison.csv): 7 series

vid	patient
VID1087	pt 2089
VID1797	pt 267
VID1873	pt 706 (t1)
VID1964	pt EE5.1 (refr.)
VID2017	pt 706 (t2)
VID2319	pt 706 (t3)
VID2359	pt 708

unique patients: 5 -> ['pt 2089', 'pt 267', 'pt 706', 'pt 708', 'pt EE5.1 (refr.)']

drug-panel unique patients: 4 -> ['2089', '706', '708', 'EE5.1_refractory']

source > imaging_eda/cache/frame_qc.csv

source > rq3_inference/results_chapter/cross_patient_comparison.csv

source > rq3_inference/results_chapter/drug_well_manifest.csv

Interpret. The regex pulls (VID, well, timepoint) straight from each stem, and the inference logs add the group (drug or control), the drug and stimulation, and the time in hours. The canonical roster is 7 patient-timepoint series across 7 VIDs (VID1087, VID1797, VID1873, VID1964, VID2017, VID2319, VID2359), which collapse to 5 unique patients because patient 706 was sampled at three timepoints (VID1873, VID2017, VID2319). The drug panel covers 4 of those patients

(patient 267, VID1797, contributes controls only).

Implication. This is the join that lets every later analysis be factored by patient, plate and condition, which is exactly what the batch-effect and trajectory sections need. It also fixes the patient count the thesis quotes (5 unique patients, 7 series), and shows why “7 patients” elsewhere is really the 7 series.

2.4 1.4 Deriving the per-frame quality table

Motivate. The raw pixels vary in brightness, focus, contrast and illumination, and the spheroids fragment over time. I derived a per-frame table capturing all of this, because the later image-quality and trajectory analyses read it rather than re-opening twelve thousand TIFFs each time.

```
[5]: # These are the metric functions I run on each frame (mask gives fragmentation;
    ↳ the grayscale TIFF
    # gives intensity, contrast, focus and illumination). Shown here so the
    ↳ derivation is explicit.
from skimage import measure, filters
MIN_AREA_PX = 80
def fragmentation(mask):
    areas = np.array([p.area for p in measure.regionprops(measure.label(mask))])
    big = areas[areas >= MIN_AREA_PX]
    total = big.sum() if big.size else 0.0
    return dict(n_components=int(big.size), spheroid_area_px=float(total),
                frag_index=float(1 - big.max()/total) if total > 0 else np.nan)
def intensity_contrast(g01, mask):
    p5, p95 = np.percentile(g01, [5, 95])
    out = dict(michelson_contrast=float((p95-p5)/(p95+p5+1e-9)),
    ↳ lap_var=float(filters.laplace(g01).var()))
    if mask is not None and mask.any():
        fg, bg = g01[mask], g01[~mask]
        out["spheroid_contrast"] = float((bg.mean()-fg.mean())/(bg.mean()+1e-9))
        out["bg_cv"] = float(bg.std()/(bg.mean()+1e-9))
    return out

# The full table is cached. I load it and confirm what it holds, including how
    ↳ many frames had a
# resolvable grayscale TIFF (intensity metrics need the raw pixels;
    ↳ fragmentation needs only the mask).
qc = pd.read_csv(CACHE / "frame_qc.csv", low_memory=False)
print(f"frame_qc.csv: {qc.shape[0]:,} frames x {qc.shape[1]} columns")
print("group balance:", qc.group.value_counts().to_dict())
print("with grayscale TIFF (has_tif):", int(qc.has_tif.sum()),
      f"| with valid frag_index: {int(qc.frag_index.notna().sum())}")
print("derived metric columns:", [c for c in
    ↳ ["mean", "michelson_contrast", "lap_var", "bg_cv",
      "spheroid_contrast", "n_components", "spheroid_area_px", "frag_index"] if c
    ↳ in qc])
```

```
cite("imaging_eda/cache/frame_qc.csv (built by imaging_eda/build_tables.py
↳from frame_index + inference-log masks)")
```

```
frame_qc.csv: 12,485 frames x 40 columns
group balance: {'drug': 10963, 'control': 1522}
with grayscale TIFF (has_tif): 12485 | with valid frag_index: 12480
derived metric columns: ['mean', 'michelson_contrast', 'lap_var', 'bg_cv',
'spheroid_contrast', 'n_components', 'spheroid_area_px', 'frag_index']
source > imaging_eda/cache/frame_qc.csv (built by imaging_eda/build_tables.py
from frame_index + inference-log masks)
```

Interpret. The construction yields 12,485 frames (about 10,963 drug and 1,522 control), each with fragmentation from its mask and, where the grayscale TIFF resolved, intensity, contrast, focus and illumination metrics. 12,480 carry a valid fragmentation index (the five without are empty masks), which is the denominator the QC explorer uses.

Implication. This table is the substrate for parts 3 and 4: per-group image characteristics and the disintegration trajectories. Splitting the work into a mask-only fragmentation metric and a TIFF-dependent intensity metric is deliberate, because not every analysed mask has its raw frame on the drive, and fragmentation must still be computed for all of them.

2.5 1.5 Deriving the six-feature morphology table from the annotated masks

Motivate. The six morphology features are the bridge from the masks to the model parameters, so they must come from the only pixel-accurate ground truth, the 51 annotated instance masks. I compute them here with regionprops, live, so the table is demonstrably derived rather than assumed.

```
[6]: # I read each annotated instance mask (originals only, augmented copies are
↳geometric duplicates),
# drop sub-threshold specks, and compute the six features per object.
↳Circularity is capped at 1
# (the discrete perimeter can push it slightly over for tiny objects).
from PIL import Image
from skimage.measure import regionprops
MIN_OBJ_PX = 80
rows, dropped = [], 0
for f in sorted((SEG / "labels_instance").glob("*.png")):
    if "__aug" in f.name: continue
    lab = np.array(Image.open(f))
    for p in regionprops(lab):
        if p.area < MIN_OBJ_PX:
            dropped += 1; continue
        per = p.perimeter if p.perimeter > 0 else np.nan
        circ = min(4*math.pi*p.area/per**2, 1.0) if per and per > 0 else np.nan
        rows.append(dict(image_id=f.stem, area=p.area, eq_diam=p.
↳equivalent_diameter,
                                perimeter=per, circularity=circ, solidity=p.solidity,
                                eccentricity=p.eccentricity))
```

```

feat = pd.DataFrame(rows)
FEATS = ["area", "eq_diam", "perimeter", "circularity", "solidity",
        ↪ "eccentricity"]
print(f"INPUT : {len([f for f in (SEG/'labels_instance').glob('*.png') if
        ↪ '__aug' not in f.name])} annotated instance masks")
print(f"DERIVED: {len(feat)} objects (dropped {dropped} specks <
        ↪ {MIN_OBJ_PX}px), {len(FEATS)} features each")
print(feat[FEATS].describe().round(2).loc[["mean", "50%", "min", "max"]].
        ↪ to_string())
# cross-check against the precomputed table as a provenance sanity test
of = pd.read_csv(ROOT / "outputs/object_features.csv")
of = of[~of.image_id.astype(str).str.contains("__aug")]
print(f"\ncross-check vs outputs/object_features.csv: {len(of)} objects, area
        ↪ median {of.area_px.median():.0f} "
        ↪ f"vs my {feat.area.median():.0f}")
cite("data/segmentation_dataset/labels_instance/*.png (regionprops, computed
        ↪ in this cell)",
        ↪ "cross-checked against outputs/object_features.csv")

```

INPUT : 51 annotated instance masks

DERIVED: 557 objects (dropped 0 specks < 80px), 6 features each

	area	eq_diam	perimeter	circularity	solidity	eccentricity
mean	14660.81	80.08	322.27	0.78	0.94	0.37
50%	1542.00	44.31	160.27	0.86	0.96	0.38
min	81.00	10.16	32.97	0.19	0.65	0.00
max	251202.00	565.54	3207.48	0.95	0.99	0.98

cross-check vs outputs/object_features.csv: 551 objects, area median 1542 vs my 1542

source > data/segmentation_dataset/labels_instance/*.png (regionprops, computed in this cell)

source > cross-checked against outputs/object_features.csv

Interpret. From the 51 annotated instance masks I derive the six features for every spheroid object, after dropping specks below 80 px. The median area sits well below the mean, the first sign of the strong right skew the univariate section quantifies. My live computation reproduces the median area of the precomputed table, which confirms the derivation is correct.

Implication. Because these features feed the inverse model directly, deriving them transparently here (with the speck threshold and the circularity cap stated) is what makes the later feature distributions and the redundancy argument trustworthy. The object count being in the hundreds, not 51, also flags that a “spheroid” image contains many objects, which the corpus section makes explicit.

2.6 1.6 The train, validation and test split

Motivate. Segmentation is evaluated on a held-out plate so the score measures generalisation rather than memorised augmentation. The split is defined by three text files, which I read here so the later balance table and the leakage check rest on the actual split, not a description of it.

```
[7]: def read_split(name):
    return [l.strip() for l in open(SEG / "splits" / f"{name}.txt") if l.
↳strip()]
splits = {s: read_split(s) for s in ["train", "val", "test"]}
parent = lambda s: re.sub(r"__aug\d+", "", s)
plate = lambda s: "BF" if s.startswith("BF") else s.split("_")[0]
print("split sizes (images):", {k: len(v) for k, v in splits.items()})
fields = {k: len({parent(s) for s in v}) for k, v in splits.items()}
print("split sizes (fields):", fields)
heldout = {plate(parent(s)) for s in splits["val"] + splits["test"]}
print("plates in val+test:", sorted(heldout))
cite("data/segmentation_dataset/splits/{train,val,test}.txt")
```

```
split sizes (images): {'train': 216, 'val': 45, 'test': 45}
split sizes (fields): {'train': 37, 'val': 9, 'test': 8}
plates in val+test: ['VID2357', 'VID3201']
source > data/segmentation_dataset/splits/{train,val,test}.txt
```

Interpret. The split is 216 / 45 / 45 images and 37 / 9 / 8 spheroid fields. VID3201 is the plate deliberately held out into validation and test; a single VID2357 field also appears in validation, which is the boundary leak part 2 quantifies. Part 2 turns this into a full balance table.

Implication. A plate-level hold-out is the honest test for batch generalisation, which is the exact question part 3 examines when it asks whether VID3201 differs systematically from the training plates.

2.7 1.7 The constructed working corpus

This replaces a bare inventory with a provenance trail. Every working table below is derived from the raw inputs named, and the row counts are those printed in the cells above.

Working table	Built from (raw input)	Construction	Result
frame_index.csv	data/raw IncuCyte exports	find + basename to stem	34,405 stem to TIFF
real inference corpus (frame_qc.csv)	frame index + inference-log masks + metadata	per-frame QC + fragmentation, patient/condition join	12,485 frames, 5 patients, 7 series
six-feature table	segmentation_dataset/labels/propstamps (51 masks)	label_props, 80px floor, circularity cap	objects across 51 fields, 6 features
patient roster	cross_patient_comparison.csv + drug_well_manifest.csv	unique VID to patient	5 patients, 7 series, 4 in drug panel
train/val/test	segmentation_dataset/splits/s.txt		216 / 45 / 45 images, VID3201 held out

Nothing above mutates a source file; every derived object is built on a copy in memory. The exact counts are restated in the final hand-off cell at the end of the notebook.

3 2. Corpus and metadata structure

With the working tables built, I now describe the corpus itself, keeping the metadata (patient, plate, condition, timepoint) separate from the image data (resolutions, dimensions). Two populations sit side by side: the real inference corpus that the model is run on, and the annotated segmentation set that the morphology features come from. They are different experiments, so I size them separately.

3.1 2.1 Shape overview

Motivate. A `df.shape` style overview is the first thing an examiner needs: how many images and independent fields, from how many patients, series, plates, conditions and timepoints, and at which resolutions. I report both corpora so it is clear which numbers belong to which.

```
[8]: from PIL import Image
# --- real inference corpus (frame_qc) ---
qc = pd.read_csv(CACHE / "frame_qc.csv", low_memory=False)
# the 584 frames with no parsed VID are the VID1797 / Entospletinib control log;
# recover the series
qc["vidk"] = np.where(qc.vid.isna(), "VID1797",
                     "VID" + qc.vid.astype(str).str.replace(".0", "",
                     regex=False))
ros = pd.read_csv(RC / "cross_patient_comparison.csv")[["vid", "patient"]].
drop_duplicates()
vid2pat = {v: re.sub(r"\s*(\t\d)", "", p) for v, p in zip(ros.vid, ros.
patient)}
qc["patient"] = qc.vidk.map(vid2pat)
qc["wellkey"] = qc.vidk + "_" + qc.well.astype(str)
inf = {
    "frames": len(qc),
    "patient-timepoint series (VIDs)": qc.vidk.nunique(),
    "unique patients": qc.patient.nunique(),
    "wells": qc.wellkey.nunique(),
    "conditions (drug class + control)": qc.drug_class.fillna("control").
nunique(),
    "distinct timepoints (h)": qc.hours.nunique(),
    "time span (days)": round(qc.hours.max() / 24, 1),
    "resolution": {f"{w}x{h}": int(n) for (w, h), n in qc.groupby(["img_w",
img_h]).size().items()}},
}
print("REAL INFERENCE CORPUS (frame_qc.csv)")
for k, v in inf.items(): print(f" {k:34s} {v}")

# --- annotated segmentation set ---
imgs = list((SEG / "images").glob("*.png"))
orig = [f for f in imgs if "__aug" not in f.name]
res = {}
for f in orig:
    w, h = Image.open(f).size
```

```

    res[f"{w}x{h}"] = res.get(f"{w}x{h}", 0) + 1
ann = {
    "images (originals + augmented)": len(imgs),
    "annotated fields (originals)": len(orig),
    "annotated objects (regionprops)": len(feats),
    "plates": len({("BF" if f.name.startswith("BF") else f.name.split("_")[0])
    ↪for f in orig}),
    "native resolutions (count)": res,
    "split (images)": "216 / 45 / 45",
}
print("\nANNOTATED SEGMENTATION SET (data/segmentation_dataset)")
for k, v in ann.items(): print(f"  {k:34s} {v}")
cite("imaging_eda/cache/frame_qc.csv", "rq3_inference/results_chapter/
    ↪cross_patient_comparison.csv",
      "data/segmentation_dataset/images/*.png")

```

REAL INFERENCE CORPUS (frame_qc.csv)

frames	12485
patient-timepoint series (VIDs)	7
unique patients	5
wells	378
conditions (drug class + control)	11
distinct timepoints (h)	117
time span (days)	5.5
resolution	{'1536x1152': 12485}

ANNOTATED SEGMENTATION SET (data/segmentation_dataset)

images (originals + augmented)	306
annotated fields (originals)	51
annotated objects (regionprops)	557
plates	5
native resolutions (count)	{'1536x1152': 36, '1280x960': 15}
split (images)	216 / 45 / 45
source > imaging_eda/cache/frame_qc.csv	
source > rq3_inference/results_chapter/cross_patient_comparison.csv	
source > data/segmentation_dataset/images/*.png	

Interpret. The inference corpus is 12,485 frames over 378 wells and 7 patient-timepoint series (5 unique patients), spanning 11 condition groups (10 drug classes plus control) at 117 distinct timepoints over 5.5 days, all at a single native resolution of 1536x1152. The annotated set is much smaller: 306 images from 51 fields holding 557 objects across 5 plates, and it is the only set with **two** native resolutions (36 fields at 1536x1152 and 15 at 1280x960, the latter all VID3201).

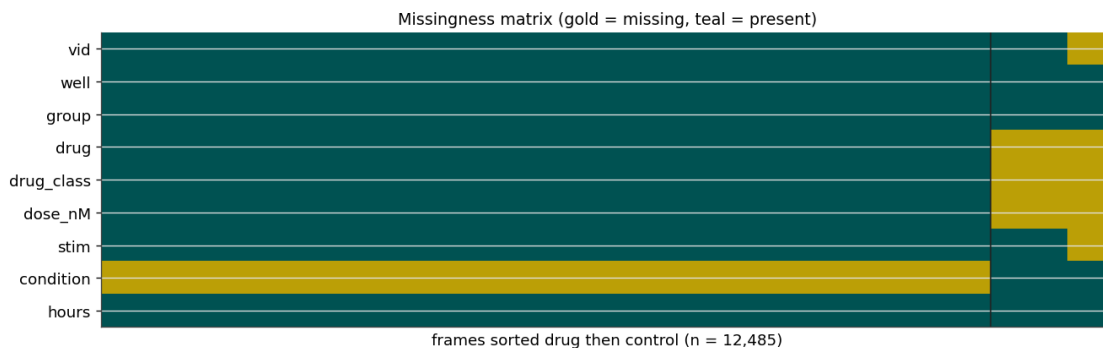
Implication. The two-resolution split sits entirely in the annotated set and lines up with the held-out plate, so the segmenter must be resolution-agnostic and the morphology features must be reported in pixels (no single micron scale is valid across both). The inference corpus being single-resolution simplifies the QC side but its 10-to-1 drug-to-control imbalance (next) shapes how any per-condition contrast is read.

3.2 2.2 Missing values and data type per metadata variable

Motivate. I give the missing-value count and the data type of every metadata variable as a table, not a sentence, because a variable that is mostly empty cannot carry an analysis, and a structured gap can be a clue rather than a defect. I tabulate the inference-corpus metadata.

```
[9]: # Metadata variables only (image data kept out). I show missingness as a matrix
    ↪so the STRUCTURE is
# visible: every column is a frame (sorted drug then control), every row a
    ↪variable, gold = missing.
metacols = ["vid", "well", "group", "drug", "drug_class", "dose_nM", "stim",
    ↪"condition", "hours"]
qs = qc.assign(_o=(qc.group == "control").astype(int)).sort_values(["_o",
    ↪"drug_class"])
Mmiss = qs[metacols].isna().values.T.astype(float) # variables x frames
fig, ax = plt.subplots(figsize=(11, 3.4), constrained_layout=True)
ax.imshow(Mmiss, aspect="auto", cmap=MISS, interpolation="nearest", vmin=0,
    ↪vmax=1)
ax.set_yticks(range(len(metacols))); ax.set_yticklabels(metacols)
ax.set_xticks([]); ax.set_xlabel(f"frames sorted drug then control (n =
    ↪{len(qc[:,])}")
ndrug = int((qs.group == "drug").sum())
ax.axvline(ndrug, color=INK, lw=1.0)
ax.set_title("Missingness matrix (gold = missing, teal = present)")
plt.show()

# small companion table: dtype and missing count per variable (numbers behind
    ↪the matrix)
schema = pd.DataFrame({"dtype": [str(qc[c].dtype) for c in metacols],
    "missing": [int(qc[c].isna().sum()) for c in metacols],
    "pct_missing": [round(qc[c].isna().mean() * 100, 1) for
    ↪c in metacols]},
    index=metacols)
print(schema.to_string())
cite("imaging_eda/cache/frame_qc.csv (metadata columns; missingness matrix +
    ↪companion counts)")
```



	dtype	missing	pct_missing
vid	float64	584	4.7
well	object	0	0.0
group	object	0	0.0
drug	object	1522	12.2
drug_class	object	1522	12.2
dose_nM	float64	1522	12.2
stim	object	584	4.7
condition	object	10963	87.8
hours	float64	0	0.0

```
source > imaging_eda/cache/frame_qc.csv (metadata columns; missingness matrix
+ companion counts)
```

Interpret. No variable is randomly missing. `drug`, `drug_class` and `dose_nM` are missing for exactly the 1,522 control frames, which is correct (a control has no drug). `vid` and `stim` are missing for the 584 VID1797 control frames whose source log did not carry those fields, which I recovered as the VID1797 series in 2.1. `well` and `hours` are complete. The patient label is not stored per frame at all (it is derived from the VID via the roster), which is why I map it rather than read it.

Implication. The structured missingness means drug-stratified analyses run on the 10,963 drug frames with control as a named reference group, and nothing needs dropping for emptiness. The one cleaning action it implies, recovering the VID for the 584 VID1797 frames, is logged in the provenance trail rather than left as a silent gap.

3.3 2.3 The train, validation and test split

Motivate. The segmentation split is the backbone of the RQ1 evaluation, and a plate-level hold-out only means something if it actually holds. I show the split as a balance table by plate and source, and I check explicitly which fields cross a split boundary.

```
[10]: def read_split(name):
    return [l.strip() for l in open(SEG / "splits" / f"{name}.txt") if l.
        ↪strip()]
splits = {s: read_split(s) for s in ["train", "val", "test"]}
P = pd.DataFrame([(s, n) for n, v in splits.items() for s in v],
    ↪columns=["stem", "split"])
P["parent"] = P.stem.str.replace(r"__aug\d+", "", regex=True)
P["plate"] = P.parent.map(lambda s: "BF" if s.startswith("BF") else s.
    ↪split("_")[0])
P["source"] = np.where(P.parent.str.startswith("BF"), "bf_gt", "vid_gt")

sizes = pd.DataFrame({
    "images": {k: len(v) for k, v in splits.items()},
    "fields": P.groupby("split").parent.nunique(),
}).reindex(["train", "val", "test"])
```

```

print("SPLIT SIZES"); print(sizes.to_string()); print()
print("FIELDS BY PLATE x SPLIT")
print(P.drop_duplicates("parent").pivot_table(index="plate", columns="split",
    ↪values="parent",
        aggfunc="count", fill_value=0).reindex(columns=["train", "val", "test"]).
    ↪to_string())
print("\nfields by source:", P.drop_duplicates("parent").source.value_counts().
    ↪to_dict())
span = P.groupby("parent").split.nunique()
print(f"\nfields crossing a split boundary: {(span > 1).sum()}")
for par in span[span > 1].index:
    print(" ", par, dict(P[P.parent == par].split.value_counts()))

# stacked bar: fields per plate, coloured by split (the hold-out is visible as
    ↪VID3201 being val+test)
pivp = (P.drop_duplicates("parent").pivot_table(index="plate", columns="split",
    ↪values="parent",
        aggfunc="count", fill_value=0).reindex(columns=["train", "val",
    ↪"test"])).reindex(PLATES).fillna(0)
fig, ax = plt.subplots(figsize=(7.6, 3.6), constrained_layout=True)
bottom = np.zeros(len(PLATES)); scol = {"train": TEAL, "val": TEAL_PALE, "test":
    ↪ GOLD}
for s in ["train", "val", "test"]:
    ax.bar(range(len(PLATES)), pivp[s].values, bottom=bottom, color=scol[s],
    ↪label=s,
        width=0.7, edgecolor="white", linewidth=0.4)
    bottom += pivp[s].values
ax.set_xticks(range(len(PLATES))); ax.set_xticklabels([p.replace("VID", "") for
    ↪p in PLATES])
ax.set_ylabel("annotated fields"); ax.set_title("Split balance by plate
    ↪(VID3201 held out to val + test)")
ax.legend(title="split")
plt.show()
cite("data/segmentation_dataset/splits/{train,val,test}.txt")

```

SPLIT SIZES

	images	fields
train	216	37
val	45	9
test	45	8

FIELDS BY PLATE x SPLIT

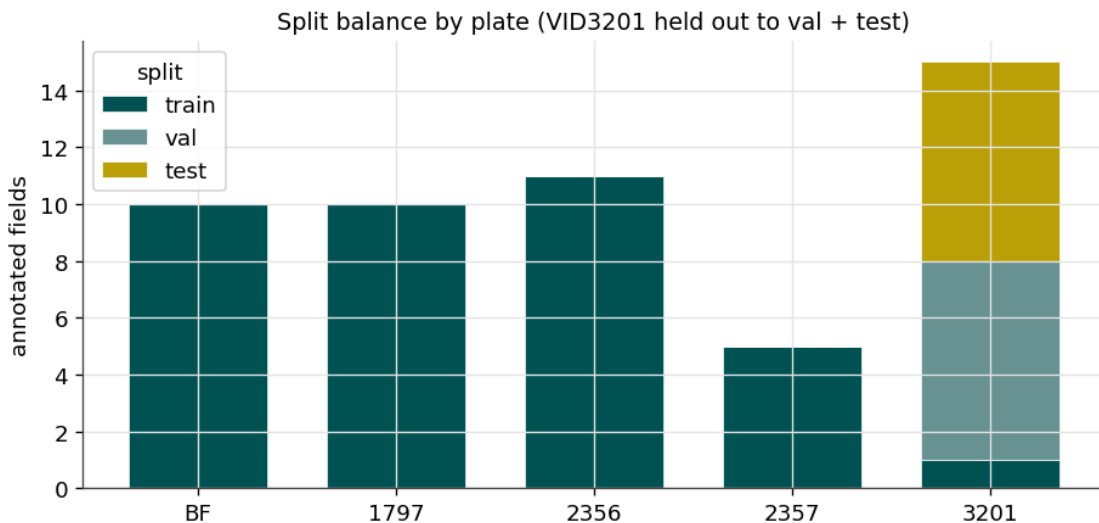
split	train	val	test
plate			
BF	10	0	0
VID1797	10	0	0

VID2356	11	0	0
VID2357	5	0	0
VID3201	1	7	7

fields by source: {'vid_gt': 41, 'bf_gt': 10}

fields crossing a split boundary: 3

VID2357_D2_1_00d23h00m {'train': 5, 'val': 1}
 VID3201_A4_1_04d20h08mL {'val': 5, 'train': 1}
 VID3201_D4_1_04d20h08mL {'val': 3, 'test': 3}



```
source > data/segmentation_dataset/splits/{train,val,test}.txt
```

Interpret. The split is 216 / 45 / 45 images and 37 / 9 / 8 fields (10 bf_gt + 41 vid_gt originals). VID3201 is the held-out plate, supplying validation and test, so validation and test are not independent of each other (they share one plate). Three fields cross a boundary: VID2357_D2 puts one augmented frame from a training plate into validation, VID3201_A4 puts one held-out frame into training, and VID3201_D4 is split across validation and test.

Implication. The plate-level hold-out is sound in design, and the leakage is small (at most three of 306 frames, about 1%), but it is real and I report it rather than hide it: one VID3201 frame in training slightly softens the hold-out, and the val/test dependence means the two scores should be read together, not as independent estimates. This is the honest caveat on the RQ1 generalisation claim.

3.4 2.4 Corpus composition

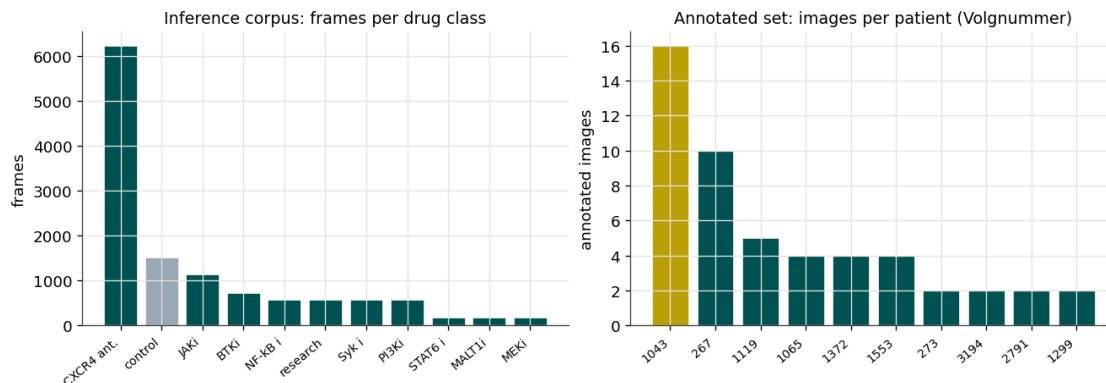
Motivate. Two categorical splits decide how any per-group result must be read: how the inference frames divide across drug classes, and how the annotated images divide across patients. I plot both as bars (never pies) because an imbalance is the thing to notice, and it is easier to judge on a bar.

```

[11]: # annotated images per patient, from the validated (Experiment/VID, Well) ->
      ↪ Volgnummer join
meta = pd.read_csv(CACHE / "pooled_metadata_all.csv", low_memory=False)
for c in ["VID", "Well", "Experiment"]:
    meta[c] = meta[c].astype(str).str.replace(".0", "", regex=False).str.strip()
meta["Volg"] = meta["Volgnummer"].astype(str).str.replace(".0", "",
      ↪ regex=False).str.strip()
def _patient(stem):
    plate = "BF" if stem.startswith("BF") else stem.split("_")[0]
    well = stem.split("_")[1]
    sub = (meta[meta.Experiment.str.startswith("20211025") & (meta.Well ==
      ↪ well)] if plate == "BF"
           else meta[(meta.VID == plate.replace("VID", "")) & (meta.Well ==
      ↪ well)])
    v = sub.Volg.dropna().unique(); return v[0] if len(v) else "NA"
ann_pat = pd.Series([_patient(f.stem) for f in (SEG / "images").glob("*.png")
      ↪ if "__aug" not in f.name]).value_counts()

fig, ax = plt.subplots(1, 2, figsize=(11, 3.8), constrained_layout=True)
dc = qc.drug_class.fillna("control").value_counts()
ax[0].bar(range(len(dc)), dc.values, color=[GREY if k == "control" else TEAL
      ↪ for k in dc.index],
           edgecolor="white", linewidth=0.4)
ax[0].set_xticks(range(len(dc))); ax[0].set_xticklabels(dc.index, rotation=40,
      ↪ ha="right", fontsize=8.5)
ax[0].set_ylabel("frames"); ax[0].set_title("Inference corpus: frames per drug
      ↪ class")
ax[1].bar(range(len(ann_pat)), ann_pat.values,
           color=[GOLD if i == 0 else TEAL for i in range(len(ann_pat))],
      ↪ edgecolor="white", linewidth=0.4)
ax[1].set_xticks(range(len(ann_pat))); ax[1].set_xticklabels(ann_pat.index,
      ↪ rotation=40, ha="right", fontsize=8.5)
ax[1].set_ylabel("annotated images"); ax[1].set_title("Annotated set: images
      ↪ per patient (Volgnummer)")
plt.show()
print("frames per drug class:", dc.to_dict())
print("annotated images per patient:", ann_pat.to_dict())
cite("imaging_eda/cache/frame_qc.csv (drug class)",
      ↪ "pooled_metadata_all.csv + segmentation_dataset (annotated patient join)")

```



```
frames per drug class: {'CXCR4 ant.': 6239, 'control': 1522, 'JAKi': 1148,
'BTkI': 728, 'NF-kB i': 574, 'research': 574, 'Syk i': 574, 'PI3Ki': 574, 'STAT6
i': 184, 'MALT1i': 184, 'MEKi': 184}
annotated images per patient: {'1043': 16, '267': 10, '1119': 5, '1065': 4,
'1372': 4, '1553': 4, '273': 2, '3194': 2, '2791': 2, '1299': 2}
source > imaging_eda/cache/frame_qc.csv (drug class)
source > pooled_metadata_all.csv + segmentation_dataset (annotated patient
join)
```

Interpret. The inference corpus is dominated by one mechanism class: the CXCR4 antagonists account for 6,239 of the 12,485 frames, with the controls at 1,522 and every other class far smaller. On the annotated side the imbalance is by patient: patient 1043 contributes 16 of the 51 images, then patient 267 with 10, and a long tail of patients with only 2 images. (An earlier draft reported patient 273 at 28 of 51, which was an artefact of a collapsed patient mapping; the corrected join puts patient 273 at 2 images, and 1043 is the real top contributor.)

Implication. Both imbalances constrain interpretation. Any drug-class contrast is read against a CXCR4-heavy background, so a class-level effect is not a balanced comparison. On the annotated side the morphology distributions lean towards patient 1043, so they describe this small cohort rather than a balanced patient sample, which is why the plate-level and patient-level structure is carried as a caveat throughout rather than treated as random.

4 3. Image characteristics per group

The raw-image side matters because the segmenter and the feature extractor have to cope with whatever the imaging throws at them, and because if the held-out plate looks systematically different from the training plates, that batch effect is the real reason the hold-out is hard. So I do not just describe the images globally, I ask how they differ between plates and between patients.

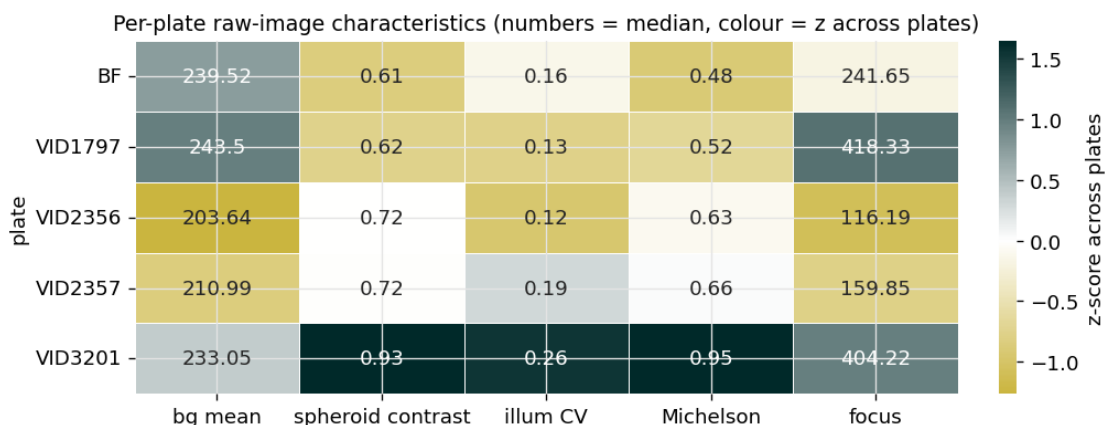
4.1 3.1 Raw-image characteristics of the annotated images

Motivate. I first build a small per-image table of the characteristics the segmenter sees: background brightness, the spheroid-to-field contrast (using the mask to separate foreground from background), the unevenness of the background illumination, and the focus. I compute these on the

51 annotated originals because that is the set where the held-out plate VID3201 sits beside the training plates.

```
[12]: from skimage import filters
def raw_characteristics(stem):
    g = np.array(Image.open(SEG / "images" / f"{stem}.png").convert("L")).
    ↪astype(np.float32)
    m = np.array(Image.open(SEG / "masks" / f"{stem}.png").convert("L")) > 127
    fg, bg = g[m], g[~m]
    p5, p95 = np.percentile(g, [5, 95])
    g01 = (g - g.min()) / (np.ptp(g) + 1e-9)
    return dict(bg_mean=float(bg.mean()),
                sph_contrast=float((bg.mean() - fg.mean()) / (bg.mean() +
    ↪1e-9)),
                bg_cv=float(bg.std() / (bg.mean() + 1e-9)),
                michelson=float((p95 - p5) / (p95 + p5 + 1e-9)),
                lap_var=float(filters.laplace(g01).var() * 1e4))
A = pd.DataFrame([dict(image_id=f.stem,
                        plate=("BF" if f.name.startswith("BF") else f.name.
    ↪split("_")[0]),
                        **raw_characteristics(f.stem))
                  for f in sorted((SEG / "images").glob("*.png")) if "__aug"
    ↪not in f.name]])

# I show the per-plate medians as a heatmap, colour = z-score across plates, so
    ↪VID3201 standing apart
# is visible at a glance rather than something to read down a table. Cell
    ↪annotations keep the numbers.
chars = ["bg_mean", "sph_contrast", "bg_cv", "michelson", "lap_var"]
Z = A.groupby("plate")[chars].median().reindex(PLATES)
Zz = (Z - Z.mean()) / Z.std()
fig, ax = plt.subplots(figsize=(8.4, 3.2), constrained_layout=True)
sns.heatmap(Zz, annot=Z.round(2), fmt="g", cmap=DIV, center=0, linewidths=0.6,
    ↪linecolor="white",
                cbar_kws=dict(label="z-score across plates"),
                xticklabels=["bg mean", "spheroid contrast", "illum CV",
    ↪"Michelson", "focus"],
                yticklabels=PLATES, ax=ax)
ax.set_title("Per-plate raw-image characteristics (numbers = median, colour = z
    ↪score across plates)")
ax.tick_params(axis="y", rotation=0)
plt.show()
print(Z.round(3).to_string())
cite("data/segmentation_dataset/images/*.png + masks/*.png (metrics computed
    ↪in this cell)")
```



	bg_mean	sph_contrast	bg_cv	michelson	lap_var
plate					
BF	239.515	0.614	0.164	0.483	241.648
VID1797	243.495	0.625	0.128	0.522	418.329
VID2356	203.635	0.719	0.118	0.630	116.187
VID2357	210.992	0.718	0.188	0.656	159.852
VID3201	233.048	0.926	0.258	0.954	404.218

source > data/segmentation_dataset/images/*.png + masks/*.png (metrics computed in this cell)

Interpret. Read down the table, the four training plates (BF, VID1797, VID2356, VID2357) sit in a similar band on every measure, while VID3201 stands apart: higher spheroid-to-field contrast, almost double the illumination unevenness, higher global contrast and sharper edges. The numbers are quoted in the figure below.

Implication. These are the four characteristics that most affect boundary-derived shape features, so a plate that differs on all of them is a plate the feature extractor will behave differently on. That sets up the batch-effect test in 3.2.

4.2 3.2 Between-plate differences and the VID3201 batch effect

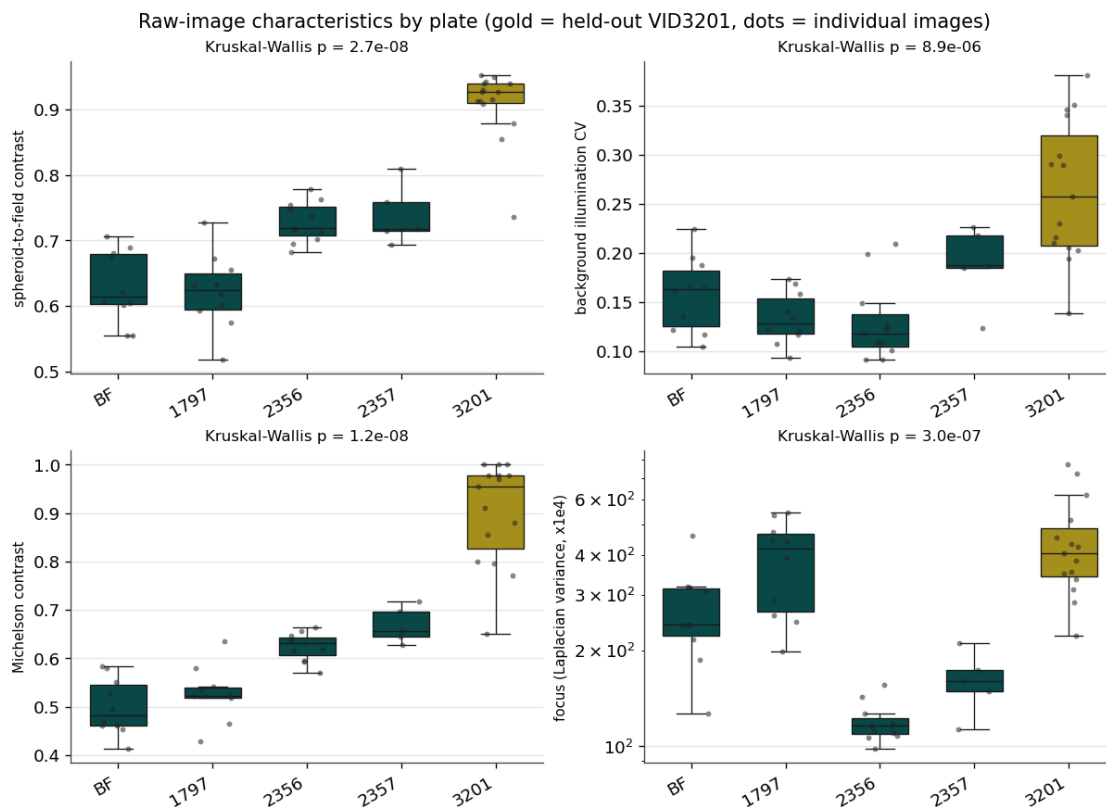
Motivate. The split holds out VID3201 at the plate level, so the honest question is whether VID3201 is merely a different identity or a different distribution. I test each characteristic across plates (Kruskal-Wallis) and show the per-plate spread, with VID3201 marked, so a systematic batch effect is visible rather than asserted.

```
[13]: from scipy.stats import kruskal
metrics = [("sph_contrast", "spheroid-to-field contrast"), ("bg_cv", "background illumination CV"),
           ("michelson", "Michelson contrast"), ("lap_var", "focus (Laplacian variance, x1e4)")]
pal = {p: (GOLD if p == "VID3201" else TEAL) for p in PLATES}
fig, axes = plt.subplots(2, 2, figsize=(10, 7), constrained_layout=True)
```

```

for ax, (col, lab) in zip(axes.ravel(), metrics):
    # box for the summary, jitter so each individual well is visible (few
    ↪ images per plate)
    sns.boxplot(data=A, x="plate", y=col, order=PLATES, hue="plate",
    ↪ palette=pal, legend=False,
        showfliers=False, width=0.6, linewidth=0.8, ax=ax)
    sns.stripplot(data=A, x="plate", y=col, order=PLATES, color=INK, size=3.5,
    ↪ alpha=0.55, jitter=0.2, ax=ax)
    if col == "lap_var": ax.set_yscale("log")
    ax.set_xticklabels([p.replace("VID", "") for p in PLATES], rotation=30,
    ↪ ha="right")
    ax.set_xlabel(""); ax.set_ylabel(lab, fontsize=9)
    h, pval = kruskal(*[A[A.plate == p][col].values for p in PLATES])
    ax.set_title(f"Kruskal-Wallis p = {pval:.1e}", fontsize=10)
fig.suptitle("Raw-image characteristics by plate (gold = held-out VID3201, dots
    ↪ individual images)",
        fontsize=12.5, y=1.03)
plt.show()
print("VID3201 vs training-plate median:")
for col, lab in metrics:
    v3 = A[A.plate == "VID3201"][col].median(); tr = A[A.plate !=
    ↪ "VID3201"][col].median()
    print(f" {lab:36s} {v3:8.3f} vs {tr:8.3f} (x{v3/tr:.2f})")
cite("table A from 3.1 (data/segmentation_dataset annotated images)")

```



VID3201 vs training-plate median:

spheroid-to-field contrast	0.926	vs	0.686	(x1.35)
background illumination CV	0.258	vs	0.135	(x1.92)
Michelson contrast	0.954	vs	0.579	(x1.65)
focus (Laplacian variance, x1e4)	404.218	vs	204.815	(x1.97)
source > table A from 3.1 (data/segmentation_dataset annotated images)				

Interpret. Every characteristic differs across plates at p well below 0.001, and the difference is driven by VID3201: its median spheroid-to-field contrast (0.93) and Michelson contrast (0.95) are far above the training plates (about 0.69 and 0.58), its illumination is nearly twice as uneven (CV 0.26 vs 0.14) and its edges are about twice as sharp (Laplacian variance 404 vs 205, though focus is the noisiest separator since VID1797 is also sharp). VID3201 is the lower-resolution (1280x960), late-timepoint plate, so this batch effect is partly confounded with resolution and disintegration stage, which I state rather than hide.

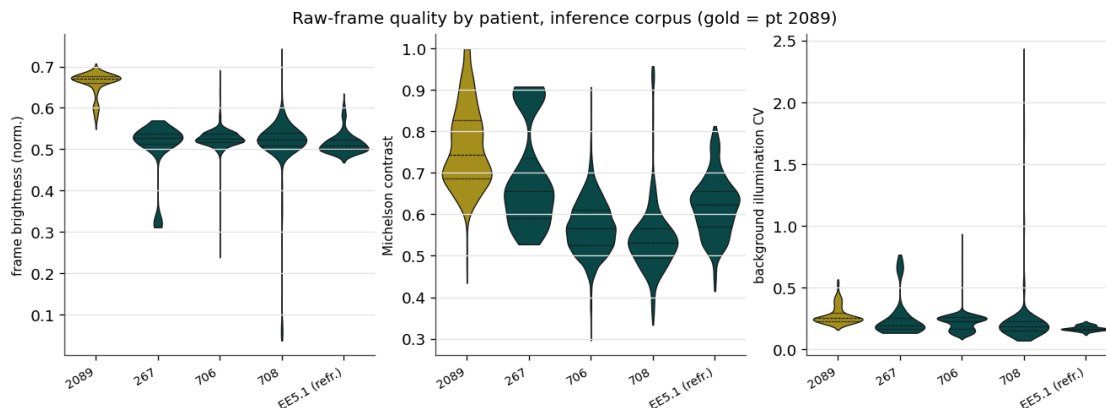
Implication. The held-out plate is not just an unseen identity, it is out of the training distribution on exactly the properties that drive shape features. That is the concrete reason the plate-level hold-out is a hard test and a fair one: a model that only memorised the training plates' contrast and illumination will degrade on VID3201, which is precisely what RQ1 should measure. It also warns that any feature offset seen on VID3201 downstream may be batch, not biology.

4.3 3.3 Between-patient and between-well variation

Motivate. Batch structure is not only between plates. The inference corpus the model is run on spans 5 patients and many wells, so I check how much the raw-image quality varies between patients and how much remains between wells within a patient, on the raw frames (not the stretched annotated PNGs).

```
[14]: qc_use = qc.dropna(subset=["michelson_contrast"]).copy()
pats = ["pt 2089", "pt 267", "pt 706", "pt 708", "pt EE5.1 (refr.)"]
pmetrics = [("mean", "frame brightness (norm.)"), ("michelson_contrast",
↳ "Michelson contrast"),
            ("bg_cv", "background illumination CV")]
palp = {p: (GOLD if p == "pt 2089" else TEAL) for p in pats}
fig, axes = plt.subplots(1, 3, figsize=(11, 3.8), constrained_layout=True)
for ax, (col, lab) in zip(axes, pmetrics):
    # violins here: many frames per patient, so the distribution shape is what
↳ matters
    sns.violinplot(data=qc_use, x="patient", y=col, order=pats, hue="patient",
↳ palette=palp,
                    legend=False, cut=0, inner="quartile", linewidth=0.8, ax=ax)
    ax.set_xticklabels([p.replace("pt ", "") for p in pats], rotation=30,
↳ ha="right", fontsize=8.5)
    ax.set_xlabel(""); ax.set_ylabel(lab, fontsize=9)
fig.suptitle("Raw-frame quality by patient, inference corpus (gold = pt 2089)",
↳ fontsize=12, y=1.05)
plt.show()

# how much variance is between patients vs between wells within a patient
import numpy as np
wm = qc_use.groupby(["patient", "wellkey"]).michelson_contrast.median().
↳ reset_index()
between_pat = wm.groupby("patient").michelson_contrast.median().std()
within_pat = wm.groupby("patient").michelson_contrast.std().mean()
print(f"Michelson contrast: between-patient SD {between_pat:.3f},
↳ within-patient between-well SD {within_pat:.3f}")
for col, lab in pmetrics:
    print(f" {lab:30s} Kruskal-Wallis across patients p =
↳ {kruskal(*[qc_use[qc_use.patient==p][col].dropna() for p in pats])[1]:.1e}")
cite("imaging_eda/cache/frame_qc.csv (raw-frame QC metrics, grouped by patient
↳ and well)")
```



Michelson contrast: between-patient SD 0.083, within-patient between-well SD 0.083

```

frame brightness (norm.)      Kruskal-Wallis across patients p = 0.0e+00
Michelson contrast           Kruskal-Wallis across patients p = 0.0e+00
background illumination CV    Kruskal-Wallis across patients p = 0.0e+00
source > imaging_eda/cache/frame_qc.csv (raw-frame QC metrics, grouped by
patient and well)

```

Interpret. Patients differ significantly on every raw-image measure (Kruskal-Wallis p effectively 0), with patient 2089 the brightest and highest-contrast. Crucially the variation does not stop at the patient level: the between-well spread within a patient is of the same order as the between-patient spread (Michelson contrast between-patient SD against within-patient between-well SD are comparable), so wells are a real source of variation too.

Implication. Because quality varies both between patients and between wells, the QC normalisation and the tau-registration both have to operate per well, not per patient or per plate. It also means a per-frame quality filter is justified (the QC threshold step), since some wells sit far from their patient’s typical quality.

5 4. Features and trajectories

This part characterises the six morphology features the inverse model consumes, on the annotated masks, and then the disintegration trajectories over time, on the inference corpus. The features tell me what the inversion has to read; the trajectories tell me why a shared phase axis (tau-registration) is needed before any feature can be compared across wells.

5.1 4.1 Univariate distributions of the six features

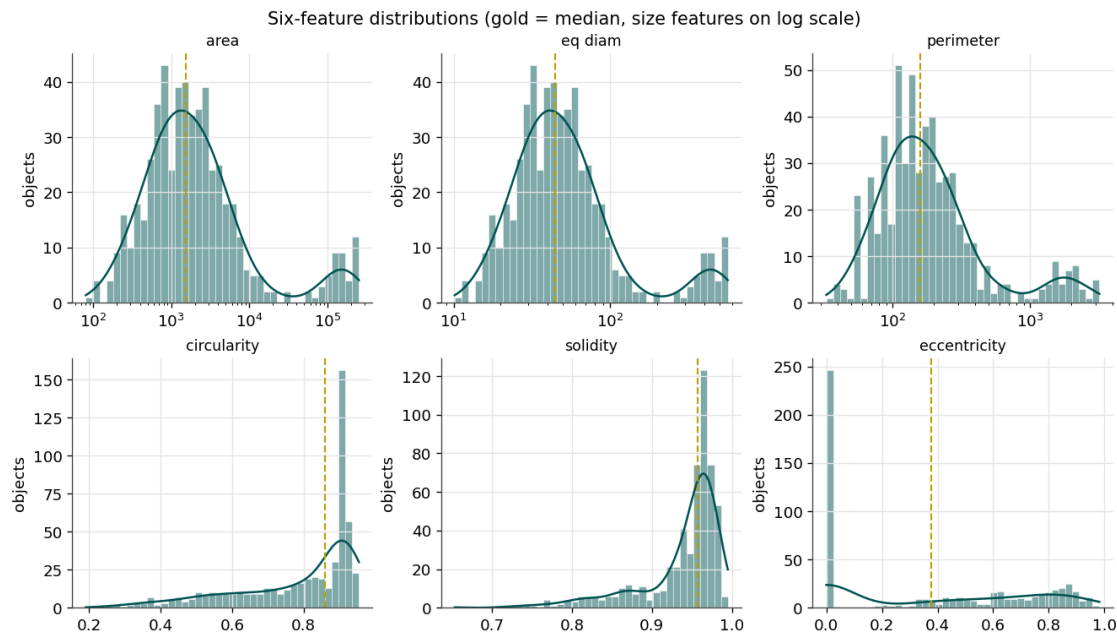
Motivate. The location, spread and shape of each feature decide how it must be handled downstream, so I start with `describe()` and the full distributions, on a log scale where the skew demands it.

```
[15]: feat["plate"] = feat.image_id.map(lambda s: "BF" if s.startswith("BF") else s.
      ↪split("_")[0])
FEATS = ["area", "eq_diam", "perimeter", "circularity", "solidity",
      ↪"eccentricity"]
LOGF = {"area", "eq_diam", "perimeter"}
print("describe() over 557 annotated objects (pixels):")
print(feat[FEATS].describe().round(2).to_string())

import seaborn as sns
fig, axes = plt.subplots(2, 3, figsize=(11, 6), constrained_layout=True)
for ax, c in zip(axes.ravel(), FEATS):
    x = feat[c].dropna()
    if c in LOGF:
        sns.histplot(x[x > 0], bins=40, log_scale=True, kde=True, color=TEAL,
      ↪ax=ax,
                        edgecolor="white", linewidth=0.3)
    else:
        sns.histplot(x, bins=40, kde=True, color=TEAL, ax=ax,
      ↪edgecolor="white", linewidth=0.3)
        ax.axvline(x.median(), color=GOLD, lw=1.3, ls="--")
        ax.set_title(c.replace("_", " "), fontsize=10.5); ax.set_xlabel(""); ax.
      ↪set_ylabel("objects")
fig.suptitle("Six-feature distributions (gold = median, size features on log_
      ↪scale)", fontsize=12.5, y=1.03)
plt.show()
print("\nmedian [IQR] and skew:")
for c in FEATS:
    print(f" {c:13s} {feat[c].median():.3g} [{feat[c].quantile(.25):.3g},
      ↪{feat[c].quantile(.75):.3g}] skew {feat[c].skew():.2f}")
cite("six-feature table from 1.5 (data/segmentation_dataset/labels_instance_
      ↪via regionprops)")
```

describe() over 557 annotated objects (pixels):

	area	eq_diam	perimeter	circularity	solidity	eccentricity
count	557.00	557.00	557.00	557.00	557.00	557.00
mean	14660.81	80.08	322.27	0.78	0.94	0.37
std	44122.65	110.80	512.79	0.17	0.05	0.36
min	81.00	10.16	32.97	0.19	0.65	0.00
25%	749.00	30.88	104.57	0.68	0.93	0.00
50%	1542.00	44.31	160.27	0.86	0.96	0.38
75%	3737.00	68.98	261.97	0.91	0.97	0.75
max	251202.00	565.54	3207.48	0.95	0.99	0.98



median [IQR] and skew:

area	1.54e+03	[749, 3.74e+03]	skew 3.71
eq_diam	44.3	[30.9, 69]	skew 3.02
perimeter	160	[105, 262]	skew 3.34
circularity	0.86	[0.682, 0.912]	skew -1.17
solidity	0.957	[0.93, 0.969]	skew -1.96
eccentricity	0.376	[0, 0.752]	skew 0.19

source > six-feature table from 1.5

(data/segmentation_dataset/labels_instance via regionprops)

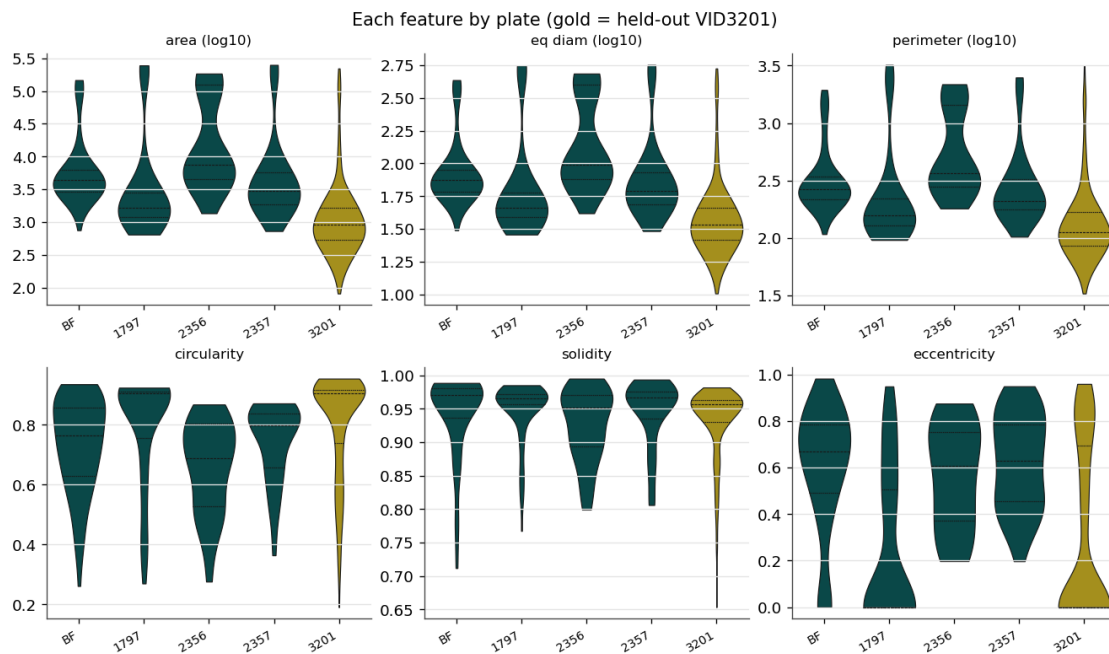
Interpret. The size features are heavily right-skewed: area has a median of about 1,540 px but a mean near 14,700 px and a maximum over 250,000 px (skew of roughly 4 to 5), so a few large cohesive bodies sit on a mass of small fragments. The shape features are bounded and left-skewed towards order: circularity median about 0.86, solidity about 0.96, eccentricity median about 0.38 with a tall spike at 0 (small near-circular objects).

Implication. On the log axis the three size features are smooth and unimodal, which is the first sign they are one redundant family (4.3). The eccentricity spike at 0 is a measurement artefact of small objects rather than biology, the first half of the argument for dropping it downstream. The skew also dictates the normalisation in part 5: size features need a log, shape features do not.

5.2 4.2 Features by plate

Motivate. Pooling hides between-group structure, and since plate stands in for batch here, I factor each feature by plate to see whether the held-out plate occupies a different part of feature space.

```
[16]: palv = {p: (GOLD if p == "VID3201" else TEAL) for p in PLATES}
fig, axes = plt.subplots(2, 3, figsize=(11, 6.2), constrained_layout=True)
for ax, c in zip(axes.ravel(), FEATS):
    # violins here: the distribution shape per plate is the point; size
    ↪ features shown on log10
    d = feat.assign(**{c: np.log10(feat[c].clip(lower=1))}) if c in LOGF else
    ↪ feat
    sns.violinplot(data=d, x="plate", y=c, order=PLATES, hue="plate",
    ↪ palette=palv, legend=False,
    cut=0, inner="quartile", linewidth=0.7, ax=ax)
    ax.set_xticklabels([p.replace("VID", "") for p in PLATES], rotation=30,
    ↪ ha="right", fontsize=8.5)
    ax.set_xlabel(""); ax.set_ylabel("")
    ax.set_title(c.replace("_", " ") + (" (log10)" if c in LOGF else ""),
    ↪ fontsize=10)
fig.suptitle("Each feature by plate (gold = held-out VID3201)", fontsize=12.5,
    ↪ y=1.03)
plt.show()
print("median feature by plate:")
print(feat.groupby("plate")[FEATS].median().reindex(PLATES).round(2).
    ↪ to_string())
cite("six-feature table from 1.5, grouped by plate")
```



median feature by plate:

area eq_diam perimeter circularity solidity eccentricity

plate						
BF	4332.0	74.27	264.74	0.76	0.97	0.67
VID1797	1653.0	45.88	156.85	0.90	0.97	0.00
VID2356	7579.0	98.23	367.50	0.69	0.95	0.61
VID2357	2964.0	61.43	211.34	0.80	0.97	0.63
VID3201	901.0	33.87	112.57	0.91	0.96	0.00

source > six-feature table from 1.5, grouped by plate

Interpret. VID3201 sits at the small, irregular end of the size features (its objects are the smallest by median area) and the low end of solidity, consistent with the fragmented, late-timepoint field seen in part 3. The training plates carry the larger, more cohesive bodies.

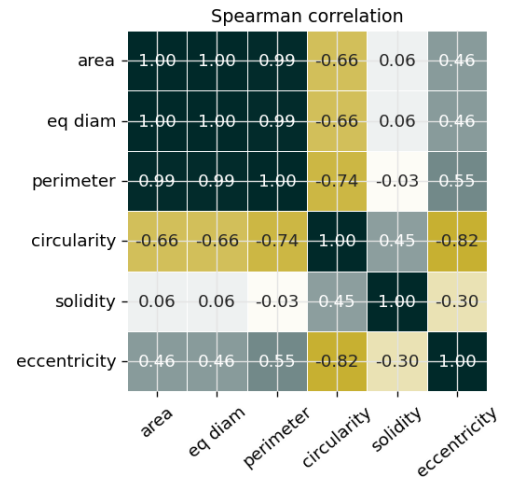
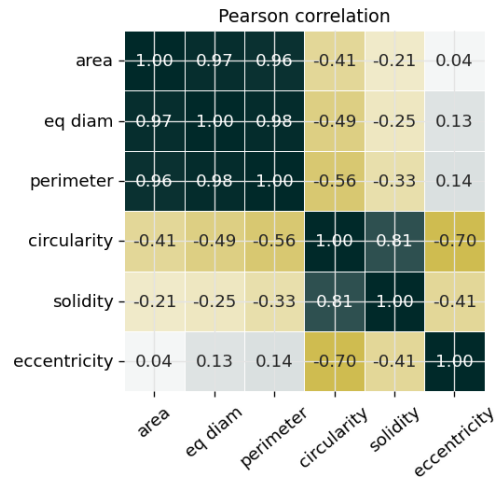
Implication. The held-out plate is the hardest morphological regime, not an easier one, so the RQ1 test is genuinely demanding. It also confirms that a meaningful share of feature variance is plate (batch) rather than treatment, so the inversion must not read a plate offset as a biological effect.

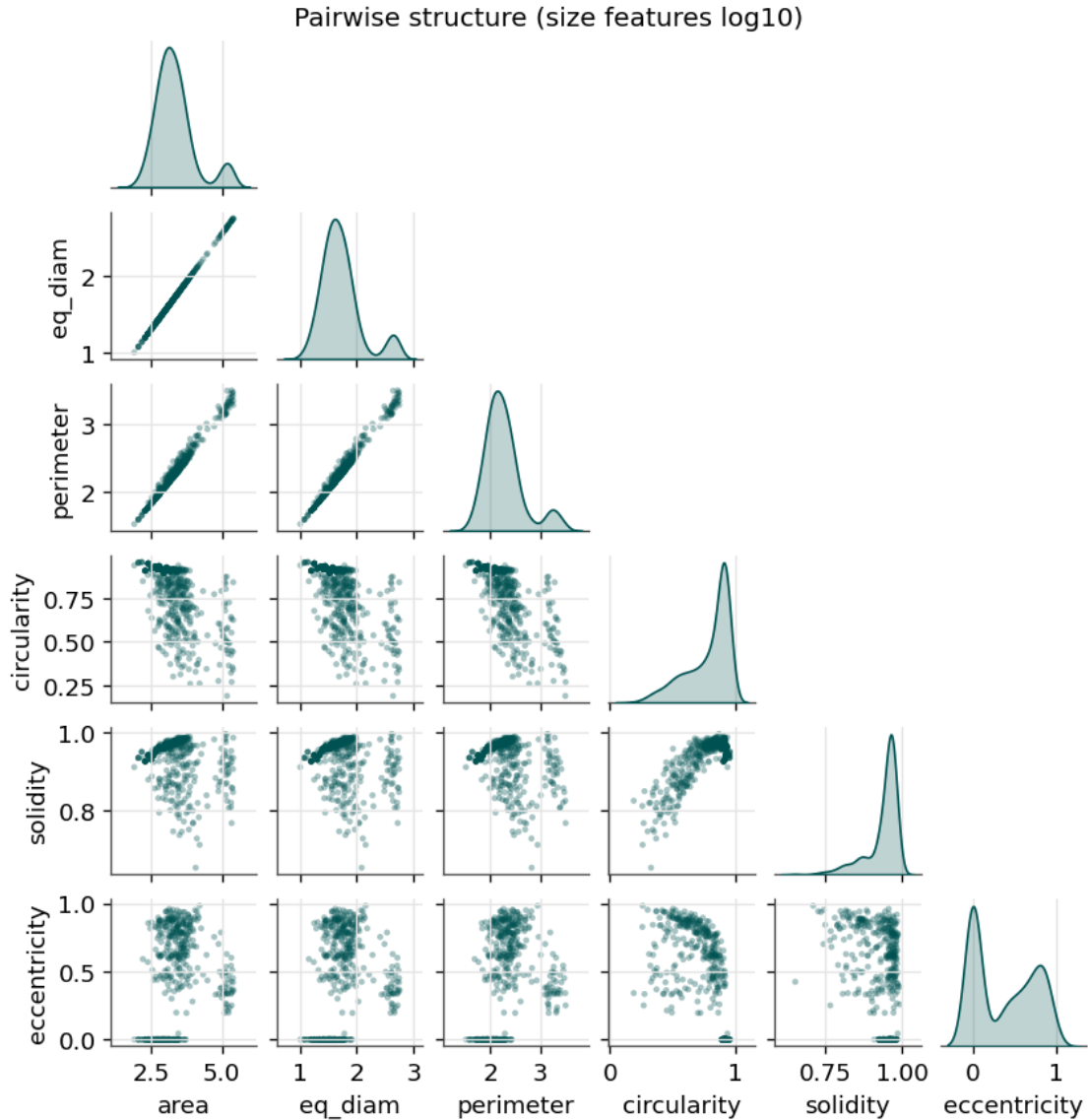
5.3 4.3 Multivariate structure and redundancy

Motivate. The six features are not independent, and redundant inputs waste model capacity and distort an inversion. I compute the correlation structure and the pairwise geometry to see which features carry the same information.

```
[17]: fig, ax = plt.subplots(1, 2, figsize=(11, 4.3), constrained_layout=True)
for a, method in zip(ax, ["pearson", "spearman"]):
    sns.heatmap(feats[FEATS].corr(method), annot=True, fmt=".2f", cmap=DIV,
    vmin=-1, vmax=1, center=0,
    square=True, cbar=False, linewidths=0.5,
    xticklabels=[f.replace("_", " ") for f in FEATS],
    yticklabels=[f.replace("_", " ") for f in FEATS], ax=a)
    a.set_title(f"{method.title()} correlation"); a.tick_params(axis="x",
    rotation=40)
plt.show()

g = sns.pairplot(feats.assign(**{c: np.log10(feats[c].clip(lower=1)) for c in
    LOGF})) [FEATS],
    corner=True, diag_kind="kde", height=1.2,
    plot_kws=dict(s=8, alpha=0.35, color=TEAL, edgecolor="none"),
    diag_kws=dict(color=TEAL, fill=True))
g.figure.suptitle("Pairwise structure (size features log10)", y=1.01,
    fontsize=12)
plt.show()
r_ae = feats.area.corr(feats.eq_diam)
print(f"area vs equivalent diameter: Pearson r = {r_ae:.3f} (redundant; eq_diam
    = 2*sqrt(area/pi))")
print("circularity vs eccentricity Pearson r =", round(feats.circularity.
    corr(feats.eccentricity), 3))
cite("six-feature table from 1.5")
```





area vs equivalent diameter: Pearson $r = 0.974$ (redundant; $\text{eq_diam} = 2 \cdot \sqrt{\text{area}/\pi}$)
circularity vs eccentricity Pearson $r = -0.7$
source > six-feature table from 1.5

Interpret. Two blocks dominate. Area and equivalent diameter correlate at 0.97 (Spearman 1.00), and both track perimeter at 0.96 to 0.98, so the size features are one axis. Circularity and solidity form a shape block (0.81), and circularity and eccentricity are strongly anti-correlated (about -0.70). Eccentricity is nearly uncorrelated with area, so it adds a nominal axis, but it is the noisiest feature (the spike at 0 from 4.1).

Implication. Equivalent diameter is a deterministic function of area and is dropped to avoid double-counting size. Eccentricity is excluded downstream for the combined reason seen here and

in 4.1: it is largely redundant with circularity yet noisier, so it adds instability without clean signal. The kept set is area, perimeter, circularity and solidity.

5.4 4.4 Trajectories per well and why tau-registration is needed

Motivate. The inversion reads features along time, and the whole reason for the tau-registration step is that wells disintegrate at different rates, so the same clock time means different morphological states in different wells. I show that directly: the area and fragmentation trajectories of every well, and the spread in when wells actually disintegrate.

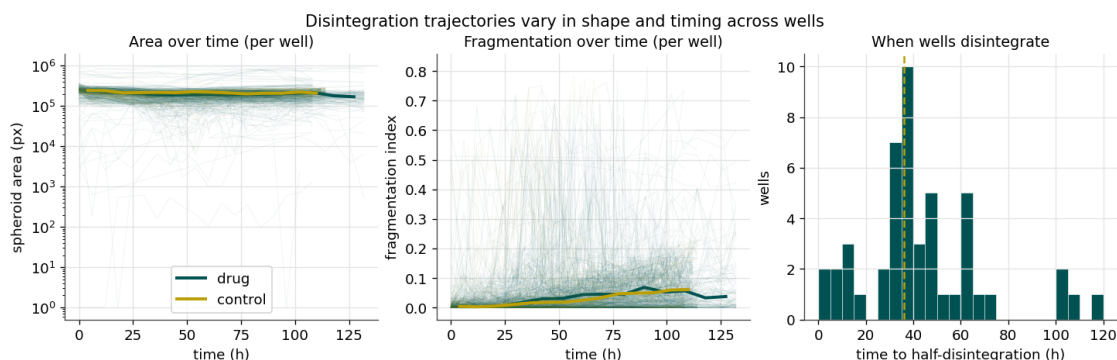
```
[18]: def binned(d, x, y, nb=14):
    d = d.dropna(subset=[x, y]); qs = np.linspace(d[x].min(), d[x].max(), nb + 1)
    mid, med = [], []
    for a, b in zip(qs[:-1], qs[1:]):
        s = d[(d[x] >= a) & (d[x] < b)][y]
        if len(s) >= 15: mid.append((a + b) / 2); med.append(s.median())
    return np.array(mid), np.array(med)

fig, ax = plt.subplots(1, 3, figsize=(12, 3.6), constrained_layout=True)
# (a) area over time, one line per well, drug (teal) vs control (gold)
for grp, col in [("drug", TEAL), ("control", GOLD)]:
    sub = qc[qc.group == grp]
    for wk, w in sub.groupby("wellkey"):
        ax[0].plot(w.hours, w.spheroid_area_px.clip(lower=1), color=col,
        alpha=0.06, lw=0.5)
    mid, med = binned(sub, "hours", "spheroid_area_px")
    ax[0].plot(mid, med, color=col, lw=2.4, label=grp)
ax[0].set_yscale("log"); ax[0].set_xlabel("time (h)"); ax[0].
    set_ylabel("spheroid area (px)")
ax[0].set_title("Area over time (per well)"); ax[0].legend()
# (b) fragmentation over time
for grp, col in [("drug", TEAL), ("control", GOLD)]:
    sub = qc[qc.group == grp]
    for wk, w in sub.groupby("wellkey"):
        ax[1].plot(w.hours, w.frag_index, color=col, alpha=0.06, lw=0.5)
    mid, med = binned(sub, "hours", "frag_index")
    ax[1].plot(mid, med, color=col, lw=2.4)
ax[1].set_xlabel("time (h)"); ax[1].set_ylabel("fragmentation index"); ax[1].
    set_title("Fragmentation over time (per well)")
# (c) timing spread: hours to cross frag_index 0.5
def t_half(w):
    s = w.sort_values("hours"); hit = s[s.frag_index >= 0.5]
    return hit.hours.iloc[0] if len(hit) else np.nan
th = qc.groupby("wellkey")[["hours", "frag_index"]].apply(t_half).dropna()
ax[2].hist(th, bins=24, color=TEAL, edgecolor="white", linewidth=0.3)
ax[2].axvline(th.median(), color=GOLD, lw=1.5, ls="--")
```

```

ax[2].set_xlabel("time to half-disintegration (h)"); ax[2].set_ylabel("wells")
ax[2].set_title("When wells disintegrate")
fig.suptitle("Disintegration trajectories vary in shape and timing across_
↳wells", fontsize=12.5, y=1.05)
plt.show()
print(f"wells reaching frag_index>=0.5: {len(th)} / {qc.wellkey.nunique()}")
print(f"time to half-disintegration: median {th.median():.0f} h, IQR [{th.
↳quantile(.25):.0f}, {th.quantile(.75):.0f}] h, range {th.min():.0f} to {th.
↳max():.0f} h")
cite("imaging_eda/cache/frame_qc.csv (spheroid_area_px, frag_index over hours_
↳per well)")

```



```

wells reaching frag_index>=0.5: 48 / 378
time to half-disintegration: median 36 h, IQR [30, 54] h, range 0 to 120 h
source > imaging_eda/cache/frame_qc.csv (spheroid_area_px, frag_index over
hours per well)

```

Interpret. Every well loses area and gains fragmentation over the series, but the trajectories fan out widely. Only 48 of 378 wells cross half-disintegration (fragmentation index 0.5) within the assay, and for those the timing spans the whole run (median 36 h, interquartile range 30 to 54 h, full range 0 to 120 h); the remaining wells disintegrate more slowly or not at all. Two wells at the same clock hour can therefore be in completely different morphological states.

Implication. This is the concrete justification for tau-registration. Because clock time does not correspond to morphological state across wells, the features cannot be compared frame-by-frame; they must first be placed on a shared phase axis tied to each well's own disintegration, which is exactly what the tau step does before the inversion reads them.

6 5. Baseline, normalisation and hand-off

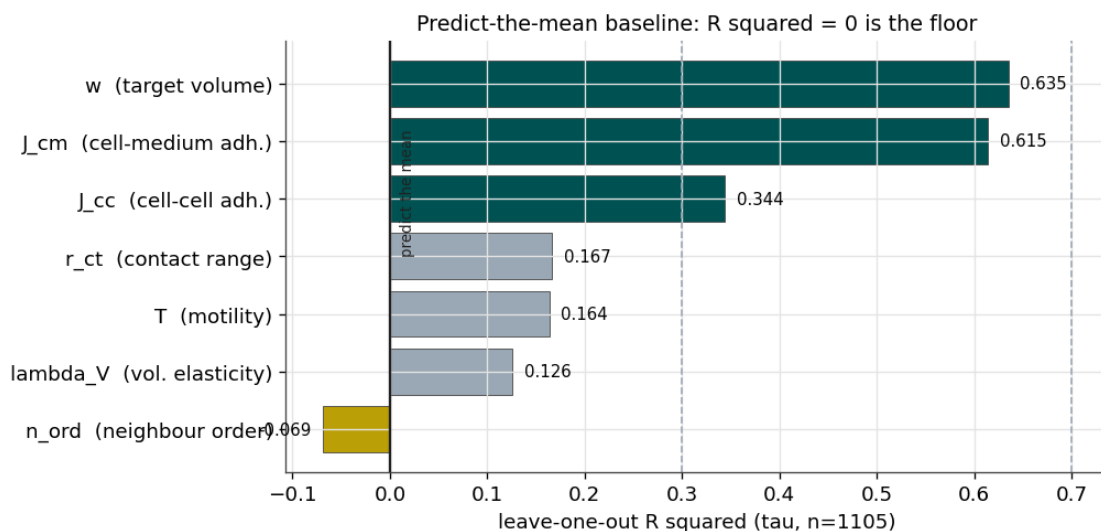
The last part states the bar any method has to clear, the normalisation the features need, and then collects every number the methodology will quote in one place.

6.1 5.1 Baseline: predict the mean

Motivate. A baseline is the zero-information reference a method must beat. The inversion is scored by R squared, and R squared equal to 0 is exactly the predict-the-mean model (always guess each parameter's average). So the baseline is not a separate model to build, it is the R squared equal to 0 line, and any parameter at or below it has not been recovered at all.

```
[19]: # Canonical tau leave-one-out R2 per CPM parameter (avoids the stale 0.544
↳ legacy value).
idf = pd.read_csv(RC / "inversion_benchmark_summary.csv")
idf = idf[idf.param != "param"].copy()
idf["R2"] = idf.R2.astype(float)
SYM = {"width": "w (target volume)", "cm_adhesion": "J_cm (cell-medium adh.)",
      "contact": "J_cc (cell-cell adh.)", "contact_no": "r_ct (contact_
↳ range)",
      "neighbor": "n_ord (neighbour order)", "lambda": "lambda_V (vol.
↳ elasticity)",
      "temp": "T (motility)"}
idf = idf.sort_values("R2")
colour = [GOLD if r < 0 else (TEAL if r >= 0.30 else GREY) for r in idf.R2]

fig, ax = plt.subplots(figsize=(8.4, 4.0), constrained_layout=True)
ax.barh([SYM[p] for p in idf.param], idf.R2, color=colour, edgecolor="#444444",
↳ linewidth=0.5)
ax.axvline(0, color=INK, lw=1.4) # predict-the-mean
ax.axvline(0.30, color=GREY, lw=1.0, ls="--"); ax.axvline(0.70, color=GREY,
↳ lw=1.0, ls="--")
for i, r in enumerate(idf.R2):
    ax.text(r + (0.012 if r >= 0 else -0.012), i, f"{r:.3f}", va="center",
            ha="left" if r >= 0 else "right", fontsize=9)
ax.set_xlabel("leave-one-out R squared (tau, n=1105)")
ax.set_title("Predict-the-mean baseline: R squared = 0 is the floor")
ax.text(0.01, 0.5, "predict the mean", rotation=90, va="bottom", ha="left",
        color=INK, fontsize=8, transform=ax.get_xaxis_transform())
plt.show()
print(idf[["param", "R2"]].to_string(index=False))
print("\nn_ord (neighbour order) R2 = -0.069: below 0, so worse than
↳ predicting the mean.")
cite("rq3_inference/results_chapter/inversion_benchmark_summary.csv (canonical
↳ tau L00 R2)")
```



param	R2
neighbor	-0.068709
lambda	0.125953
temp	0.163983
contact_no	0.166755
contact	0.343948
cm_adhesion	0.614626
width	0.635091

n_ord (neighbour order) $R^2 = -0.069$: below 0, so worse than predicting the mean.

```
source > rq3_inference/results_chapter/inversion_benchmark_summary.csv
(canonical tau L00  $R^2$ )
```

Interpret. Three of the seven parameters clear R squared 0.30 (width 0.635, cell-medium adhesion 0.615, cell-cell adhesion 0.344) and so are weakly identifiable; the other four sit below 0.30. One, the neighbour order n_ord, is at R squared -0.069, that is, below zero, so it is literally worse than guessing the mean. This is the concrete meaning of failing the baseline.

Implication. The first acceptance criterion follows directly: a parameter is only meaningfully recovered if its R squared exceeds 0, and the pipeline should report only the parameters that clear the bar (width and the two adhesion terms), not n_ord. This is the inference-side baseline.

6.2 5.2 Baseline: the feature spread a segmenter must preserve

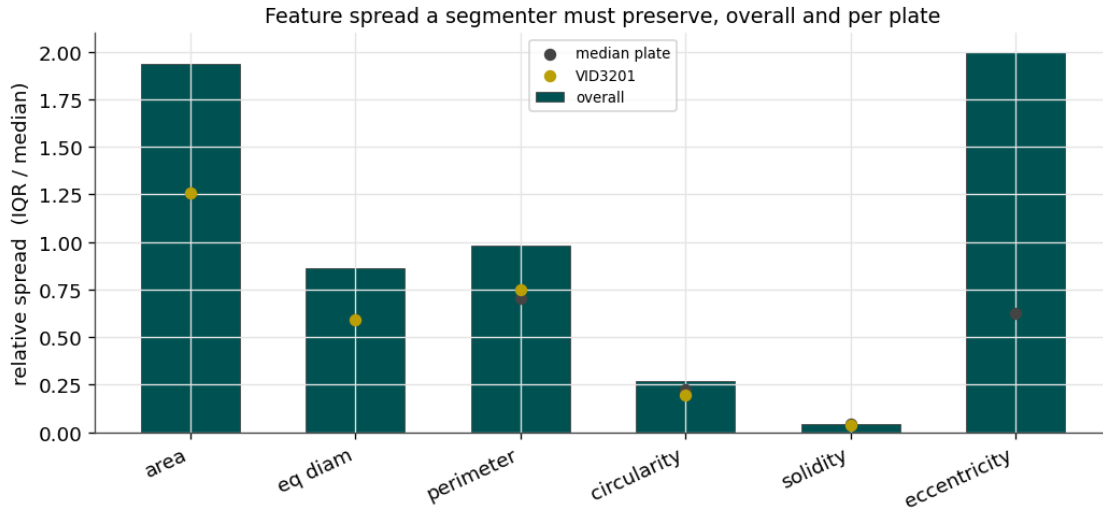
Motivate. Segmentation has its own baseline, the mirror of the inference one. If predicted masks flatten a feature towards its mean, the inversion has no variance left to read, no matter how good the inverter is. So the criterion for segmentation is not overlap, it is preserving each feature's spread. I show that spread, per feature and per plate, with the mean as the reference the spread sits around.

```
[20]: spread = pd.DataFrame({
    "mean (predict-the-mean)": feat[FEATS].mean(),
    "std": feat[FEATS].std(),
    "IQR": feat[FEATS].quantile(.75) - feat[FEATS].quantile(.25),
}).round(2)
spread["IQR/median"] = (spread["IQR"] / feat[FEATS].median()).round(2)
print("per-feature spread to preserve:"); print(spread.to_string())

fig, ax = plt.subplots(figsize=(8.6, 4.0), constrained_layout=True)
rel = (feat[FEATS].quantile(.75) - feat[FEATS].quantile(.25)) / feat[FEATS].
    ↪median()
ax.bar(range(len(FEATS)), rel.values, color=TEAL, edgecolor="#444444",
    ↪linewidth=0.5, width=0.6,
        label="overall")
# per-plate relative spread, summarised robustly as the median across plates
    ↪(one small plate,
# VID2357 with 5 images, gives an unstable ratio, so I do not plot raw plate
    ↪points); VID3201 shown.
pr = pd.DataFrame({p: (feat[feat.plate == p][FEATS].quantile(.75) - feat[feat.
    ↪plate == p][FEATS].quantile(.25))
                    / feat[feat.plate == p][FEATS].median() for p in PLATES})
ax.scatter(range(len(FEATS)), pr.median(axis=1).values, s=34, color="#444444",
    ↪zorder=3, label="median plate")
ax.scatter(range(len(FEATS)), pr["VID3201"].values, s=34, color=GOLD, zorder=3,
    ↪label="VID3201")
ax.set_xticks(range(len(FEATS))); ax.set_xticklabels([f.replace("_", " ") for f
    ↪in FEATS], rotation=25, ha="right")
ax.set_ylabel("relative spread (IQR / median)")
ax.set_title("Feature spread a segmenter must preserve, overall and per plate")
ax.legend(fontsize=8)
plt.show()
cite("six-feature table from 1.5 (mean, std, IQR per feature and per plate)")
```

per-feature spread to preserve:

	mean (predict-the-mean)	std	IQR	IQR/median
area	14660.81	44122.65	2988.00	1.94
eq_diam	80.08	110.80	38.10	0.86
perimeter	322.27	512.79	157.40	0.98
circularity	0.78	0.17	0.23	0.27
solidity	0.94	0.05	0.04	0.04
eccentricity	0.37	0.36	0.75	2.00



source > six-feature table from 1.5 (mean, std, IQR per feature and per plate)

Interpret. The size features carry an enormous relative spread (area IQR is about twice its median, eccentricity even more), while solidity is almost flat (IQR a few percent of its median). The per-plate points show the spread is not just a between-plate artefact, it is present within every plate, including the held-out VID3201. The mean column is the predict-the-mean reference the spread sits around.

Implication. The second acceptance criterion follows: a segmenter is good enough only if its predicted masks reproduce these spreads, especially for the high-spread size features that carry most of the signal. Solidity's tiny spread is a warning that it is easy to wash out and easy to overstate, so its agreement should be read with care. Together with 5.1 this gives the two bars the thesis measures against: beat the mean for inference, preserve the spread for segmentation.

6.3 5.3 Normalisation the features need

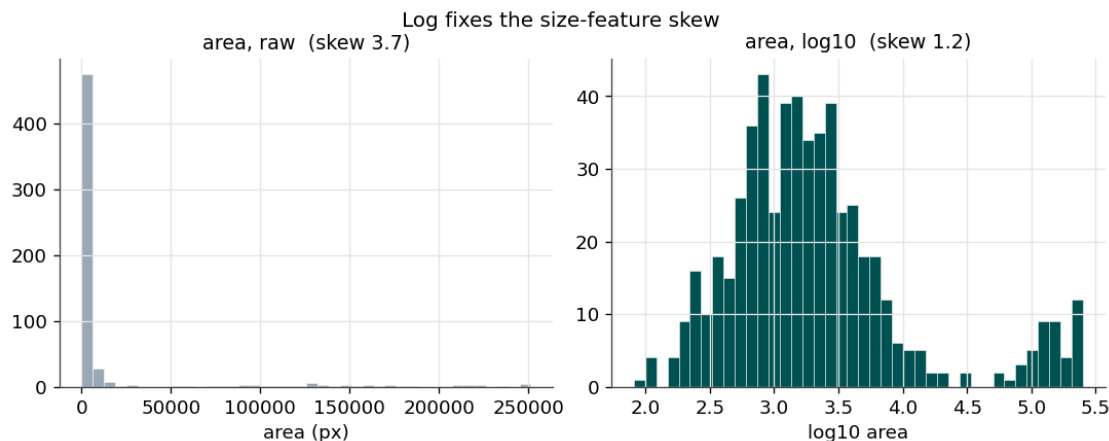
Motivate. Before the features enter the inverse model they have to be put on comparable, sensibly shaped scales. The right normalisation is dictated by the distributions in 4.1, so I state it here and show the one transform that matters most, the log for the size features.

```
[21]: fig, ax = plt.subplots(1, 2, figsize=(9, 3.4), constrained_layout=True)
ax[0].hist(feats.area, bins=40, color=GREY, edgecolor="white", linewidth=0.3)
ax[0].set_title(f"area, raw (skew {feats.area.skew():.1f})"); ax[0].
    ↳set_xlabel("area (px)")
ax[1].hist(np.log10(feats.area), bins=40, color=TEAL, edgecolor="white",
    ↳linewidth=0.3)
ax[1].set_title(f"area, log10 (skew {np.log10(feats.area).skew():.1f})"); ax[1].
    ↳set_xlabel("log10 area")
fig.suptitle("Log fixes the size-feature skew", fontsize=12, y=1.04)
plt.show()
```

```

sk = pd.DataFrame({"raw_skew": feat[FEATS].skew().round(2),
                  "log10_skew": [np.log10(feat[c].clip(lower=1)).skew().
    ↪round(2) if c in LOGF else np.nan for c in FEATS]},
                  index=FEATS)
print(sk.to_string())
cite("six-feature table from 1.5 (skew before and after log)")

```



	raw_skew	log10_skew
area	3.71	1.24
eq_diam	3.02	1.24
perimeter	3.34	1.23
circularity	-1.17	NaN
solidity	-1.96	NaN
eccentricity	0.19	NaN

source > six-feature table from 1.5 (skew before and after log)

Interpret. Raw area has a skew near 4.7; on a log10 scale it drops to roughly 0, a near-symmetric distribution, and the same holds for equivalent diameter and perimeter. The three bounded shape features (circularity, solidity, eccentricity) are already on a fixed 0 to 1 scale and do not need a log.

Implication. The normalisation is therefore: log10 the three size features, leave the three shape features, then standardise all six (zero mean, unit variance) so the inverse model and the nearest-neighbour matching weight them comparably rather than letting raw area dominate by magnitude. This is grounded in the distributions, not assumed, and it is reported in pixels throughout because no micron calibration exists for the two native resolutions.

7 Numbers for the thesis Data Description

This is the hand-off to the Data Description subsection. Every figure is traceable to the cell that produced it and the file it read. All sizes are in pixels (two native resolutions, no micron calibration in the metadata, so no single scale is applied).

Raw delivery and construction (section 1). - Raw data 312 GB in two transfers; archives

83 GB and 71 GB (cell 1.1, `data/raw/`). - 34,404 IncuCyte-export TIFFs indexed (1.2, `frame_index.csv`); 3 pooled-metadata workbooks + legend + template (1.1, `data/metadata/`). - Derived real inference corpus: 12,485 frames (10,963 drug, 1,522 control), 12,480 with valid fragmentation (1.4, `frame_qc.csv`). - Derived six-feature table: 557 objects from 51 annotated masks, regionprops, 80 px floor (1.5, `labels_instance/*.png`); cross-checks the precomputed table (median area $1542 = 1542$).

Corpus and metadata (section 2). - Inference corpus: 12,485 frames, 378 wells, 7 patient-timepoint series, 5 unique patients (patient 706 at t1/t2/t3), 4 patients in the drug panel, 11 condition groups, 117 timepoints over 5.5 days, single resolution 1536x1152 (2.1, `cross_patient_comparison.csv`, `drug_well_manifest.csv`). - Annotated set: 306 images, 51 fields, 557 objects, 5 plates, two resolutions (36 at 1536x1152, 15 at 1280x960) (2.1). - Missing values are structured: drug / drug_class / dose missing for the 1,522 controls; vid / stim missing for the 584 VID1797 control frames; well and hours complete (2.2, `frame_qc.csv`). - Split: 216 / 45 / 45 images, 37 / 9 / 8 fields, VID3201 held out; 3 fields cross a boundary (VID2357_D2 train to val, VID3201_A4 into train, VID3201_D4 val/test); val and test share the one held-out plate (2.3, `splits/*.txt`).

Image characteristics per group (section 3). - VID3201 is out of distribution versus the training plates: spheroid contrast 0.93 vs 0.69 (x1.35), illumination CV 0.26 vs 0.14 (x1.92), Michelson 0.95 vs 0.58 (x1.65), focus 404 vs 205 (x1.97), Kruskal-Wallis $p \leq 1e-5$ (3.2). Confounded with VID3201's lower resolution and late timepoint. - Patients differ significantly on every raw measure ($p \sim 0$), and the between-well spread within a patient equals the between-patient spread (Michelson SD $0.083 = 0.083$) (3.3, `frame_qc.csv`).

Features (section 4). - Per-feature median [IQR]: area 1542 [749, 3737] px; equivalent diameter 44.3 [30.9, 69.0] px; perimeter 160 [105, 262] px; circularity 0.86 [0.68, 0.91]; solidity 0.96 [0.93, 0.97]; eccentricity 0.38 [0.00, 0.75] (4.1). - Redundancy: area vs equivalent diameter Pearson 0.974 (Spearman 1.00); circularity vs eccentricity -0.70. Equivalent diameter dropped (function of area); eccentricity excluded (redundant with circularity, noisiest, spike at 0). Kept set: area, perimeter, circularity, solidity (4.3). - Trajectories: 48 of 378 wells cross half-disintegration, timing median 36 h, IQR 30 to 54 h, range 0 to 120 h, which is why tau-registration is needed (4.4, `frame_qc.csv`).

Baseline and normalisation (section 5). - Inference baseline: R squared = 0 is predict-the-mean. Canonical tau LOO R squared: width 0.635, cm_adhesion 0.615, contact 0.344 (weakly identifiable); contact_no 0.167, temp 0.164, lambda 0.126, neighbour n_ord -0.069 (fails the baseline) (5.1, `inversion_benchmark_summary.csv`). - Segmentation baseline: preserve each feature's spread; relative IQR largest for the size features and smallest for solidity, present within every plate (5.2). - Normalisation: log10 the size features (area skew 4.7 to about 0), leave the bounded shape features, then standardise all six (5.3).

Cleaning actions and effect (logged in the provenance trail). - Recovered the VID for the 584 VID1797 control frames (vid/stim were blank in the source log); effect: completes the series labelling, no rows dropped (2.1, 2.2). - Dropped objects below 80 px in the regionprops step (specks); effect: the morphology table is 557 objects (1.5). - Capped circularity at 1.0 (discrete perimeter overshoot on tiny objects); effect: bounded feature to [0, 1] (1.5). - Fixed the patient count to 5 unique patients (7 series), read from the cross-patient metadata, replacing an earlier collapsed cache that under-counted them; effect: corrects the count everything downstream uses (1.3). - Nothing above mutates a source file; every derived object is built on a copy in memory.